Smart Waste Management System: A Desktop Application with Waste Detection and Database-Driven Analytics

Md. Muradul Islam

ID# 22203234

A Practicum in the Partial Fulfillment of the Requirements

for the Award of Bachelor of Computer Science and Engineering (BCSE)

Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Summer 2025

Smart Waste Management System: A Desktop Application with Waste Detection and Database-Driven Analytics

Md. Muradul Islam

A Practicum in the Partial Fulfillment of the Requirements for the Award of Bachelor of Computer Science and Engineering (BCSE)

The practicum has been examined and approved,

_____

Prof. Dr. Utpal Kanti Das
Dean and Chairman

_____

Shahinur Alam
Co-supervisor, Coordinator and Assistant Professor

_____

Md. Khairul Islam
Supervisor and Lecturer

Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Summer 2025

Letter of Transmittal

10 May 2026
The Chair
Practicum Defense Committee
Department of Computer Science and Engineering
IUBAT—International University of Business Agriculture and Technology
4 Embankment Drive Road, Sector 10, Uttara Model Town
Dhaka 1230, Bangladesh.

Subject: Letter of Transmittal.

Dear Sir,

With due respect, I am submitting my practicum report titled "Smart Waste Management System: A Desktop Application with Waste Detection and Database-Driven Analytics" as a partial requirement for the Bachelor of Computer Science and Engineering degree at IUBAT. The work covers the analysis, design, and implementation of a Python-based desktop application that supports authenticated waste detection workflows, role-based user management, alert configuration, analytics dashboards, and PDF report generation. A placeholder detection module has been integrated to allow end-to-end testing of the application before the YOLOv8 model is connected in a later phase.

I would be happy to receive any valuable feedback you have. Thank you for allowing me to contribute to this important project.
Yours sincerely,

_____

Md. Muradul Islam
22203234

Organization's Certificate

Organizational Certificate

AIQNIC

Date: 10 May, 2026

This is to certify that Md. Muradul Islam, who is pursuing B.Sc. in Computer Science from International University of Business, Agriculture and Technology, Dhaka, did his internship from 01 February 2026 and expected end date is 31st May 2026 (40 hrs/week).

As a Jr. Artificial Intelligence Engineer (Intern), his major responsibilities were:

- Developing and training AI/ML models.
- Data preprocessing and dataset management.
- Model evaluation and performance optimization.
- Assisting in computer vision-related tasks.
- Collaboration with the development team for AI integration and testing.

Warm Regards,

MD Shaiful Islam Bapin
Chief Executive Officer, AIQNIC

House-B177, Road-11,
Jahangirnagar co-operative
housing society
Savar, Dhaka-1343
01677386036
contact@aiqnic.com

Student's Declaration

I, Muradul Islam, declare that the work presented in this practicum report titled "Smart Waste Management System: A Desktop Application with Waste Detection and Database-Driven Analytics" is the result of my own effort. The analysis, design, code, and written discussion were prepared by me during the practicum period under the guidance of my supervisor and the co-supervisor named in the approval page. Where I have drawn on the work of others, I have cited the source in the body of the report and listed the full reference at the end.

I confirm that no part of this report contains plagiarized material or fabricated data. I take full responsibility for the accuracy of the work submitted, and I understand that any breach of academic integrity will be subject to the rules of the Department of Computer Science and Engineering, International University of Business Agriculture and Technology(IUBAT).

_____

Md. Muradul Islam

22203234

Supervisor's Certification

This is to certify that the practicum report titled "Smart Waste Management System: A Desktop Application with Waste Detection and Database-Driven Analytics" submitted by Muradul Islam, bearing ID# 22203234, has been carried out under my supervision in partial fulfillment of the requirements for the award of Bachelor of Computer Science and Engineering at IUBAT—International University of Business Agriculture and Technology.

To the best of my knowledge, the work is original. The student has followed the standards expected of a final-year practicum and has shown the technical understanding, analytical reasoning, and writing quality required of the program. I recommend that the report be accepted for evaluation by the practicum defense committee.

_____

Md. Khairul Islam
Supervisor and Lecturer
Department of Computer Science and Engineering
IUBAT—International University of Business Agriculture and Technology

Abstract

Solid waste management in Bangladesh faces persistent operational gaps: missed collections, unsegregated waste streams, and limited visibility into the categories and volumes of waste generated by a facility on any given day. Manual logging is slow, error-prone, and difficult to audit. The Smart Waste Management System, abbreviated as SWMS, is a Windows desktop application built to address these gaps at the facility level. The system records every detection event in a structured database, enforces role-based access for administrators, supervisors, and operators, and gives each role a workspace tuned to the actions they are allowed to perform.

The work documented in this report covers the full software engineering lifecycle for the desktop application shell. Requirements were gathered from the project supervisor, from a review of similar systems in the literature, and from the practical needs of a small recycling operation. The system was designed using activity diagrams, swimlane diagrams, sequence diagrams, CRC cards, and a class diagram. Risks were identified and tracked using an RMMM plan, the schedule was estimated using function points, and costs were summed across personnel, hardware, software, and other categories. The implementation uses Python 3.10 with PyQt5 for the user interface, SQLAlchemy on SQLite for persistence, bcrypt for password hashing, Matplotlib for embedded charts, and ReportLab for PDF generation. A placeholder detection engine was implemented so the application can be tested end-to-end before the YOLOv8 model is integrated in a later phase. The placeholder produces realistic category labels, confidence scores, and bin-fill levels, and writes annotated result images to disk.

Six representative test cases were executed against the build, covering authentication, mock detection, history filtering, alert triggering, report generation, and user management. All cases passed against their expected results. The remaining work, namely the integration of a trained computer vision model, a live camera feed, and scheduled reporting, has been deferred to a follow-up phase and is described in the limitations and future work sections. The deliverable is a stable, well-structured application that gives a small waste management operation a usable digital record from day one and a clear path to automated detection later.

Acknowledgments

All praise to Almighty Allah, who gave me the strength and patience needed to complete this practicum within the time available.

I am grateful to my supervisor, Md. Khairul Islam, for the careful reviews and the steady stream of practical advice that shaped the work. The pace of the project would not have been possible without that guidance. I also thank Shahinur Alam, the practicum coordinator, for his help with the administrative process and his prompt responses to every query I raised. Prof. Dr. Utpal Kanti Das, Chairman of the Department of Computer Science and Engineering, was generous with his time and attention, and I benefited from his suggestions during the milestone reviews.

I want to thank my classmates who joined informal discussions about the system design, particularly the early sessions on the alert rule engine and the report layout. Their questions made me clarify ideas I had taken for granted. Finally, I thank my family for their patience during the long evenings of writing and debugging. None of this work would have been finished without their support.

Table of Contents

Letter of Transmittal iii
Organization's Certificate iv
Student's Declaration v
Supervisor's Certification vi
Abstract vii
Acknowledgments viii
List of Figures xiii
List of Tables xiv
Chapter 1. Introduction 1
1.1 Introduction 2
1.2 Background of the Study 2
1.3 Methodology 3
1.3.1 Primary Sources. 3
1.3.2 Secondary Sources. 3
1.4 Objectives 4
1.4.1 Broad Objective. 4
1.4.2 Specific Objective. 4
1.5 Process Model 5
1.6 Feasibility Study 6
1.6.1 Technical Feasibility. 6
1.6.2 Economic Feasibility. 7
1.6.3 Operational Feasibility. 7
1.7 Structure of the Report 7
Chapter 2. Organizational Overview 8
2.1 Organization Overview 9
2.2 Organization Vision 9
2.3 Organization Mission 9
2.4 Organization Services 9
2.5 Organizational Structure 10

2.6 My Position in this Organization 1
2.7 Address of the Organization 11
Chapter 3. Requirement Engineering 12
3.1 Requirement Engineering 13
3.2 Requirement Elicitation 13
3.3 Requirement Analysis 14
3.4 Requirement Specifications 14
3.4.1. User Requirements 15
3.4.2. System Requirements 15
3.4.3. Functional Requirements 22
3.4.4. Non-Functional Requirements 23
3.5 Use Case Diagram 23
Chapter 4. Analysis Modeling 25
4.1 Activity Diagram 26
4.2 Swimlane Diagram 29
4.3 Sequence Diagram 32
4.4 CRC Cards 35
4.5 Class Diagram 39
Chapter 5. Risk Management 40
5.1 Risk Management 41
5.2 Risk Identification 41
5.3 Risk Analysis 42
5.4 Risk Planning 42
5.4.1. RMMM (Risk Mitigation Management and Monitoring) Planning 43
Chapter 6. Project Planning and Scheduling 48
6.1 Project planning and scheduling 49
6.2 Function Point Estimation 49
6.2.1. Functionality, Input, Output 49
6.2.2. Identify Complexity 50
6.2.3. Unadjusted Function Point Contribution 51
6.2.4. Calculation of Total Degree of Influence (TDI) 52

6.2.5. Final Calculation 53
6.3 Project Scheduling 53
Chapter 7. Project Cost Estimation 55
7.1 Project Cost Estimation.....56
7.2 Personnel Cost 56
7.3 Hardware Cost 57
7.4 Software Cost.....57
7.5 Other Cost 58
7.6 Accounts Table 58
Chapter 8. Designing 69
8.1 Data Flow Diagram.....60
8.1.1. Level 0 DFD 60
8.1.2. Level 1 DFD 61
8.1.3. Level 2 Process 1 DFD 62
8.1.4. Level 2 Process 2 DFD 62
8.2 Entity Relationship (ER) Diagram.....63
8.3 Database Field Design 64
8.4 Interface Design.....64
8.4.1. Login Screen 65
8.4.2. Dashboard 65
8.4.3. Detection Screen 66
8.4.4. History Screen 66
8.4.5. User Screen (Admin Only) 67
8.4.6. Alerts Screen 67
8.4.7. Report Screen.....68
8.4.8. Settings Screen (Admin Only) 68
Chapter 9. Testing.....69
9.1 Test Cases 70
Chapter 10. Ethical Consideration and Sustainability.....78
10.1 Ethical Considerations 79
10.1.1. Data Privacy and Security.....79

10.1.2. Intellectual Property 79
10.1.3. User Impact 79
10.1.4. Bias and fairness 80
10.1.5. Responsibility to Stakeholders 80
10.1.6. Professional Responsibility 80
10.2 Sustainability Through Software Engineering 80
10.2.1. Economical Sustainability 80
10.2.2. Social Sustainability 81
Chapter 11. Conclusion 82
11.1 Conclusion 83
11.2 Limitation 83
11.3 Future Plan 83
References 85

List of Figures

Figure 1.1 Iterative-Incremental Process Model.....6
Figure 2.1 Organizational Structure.....11
Figure 3.1 Use Case Diagram of SWMS. 24
Figure 4.1 Activity Diagram: User Authentication.....26
Figure 4.2 Activity Diagram: Waste Detection 27
Figure 4.3 Activity Diagram: Detection Verification. 28
Figure 4.4 Activity Diagram: Report Generation. 29
Figure 4.5 Swimlane Diagram: End-to-End Detection Workflow. 30
Figure 4.6 Swimlane Diagram: Alert Rule Lifecycle 31
Figure 4.7 Swimlane Diagram: User Management.....32
Figure 4.8 Sequence Diagram: Detection Process. 33
Figure 4.9 Sequence Diagram: Alert Trigger Flow. 34
Figure 4.10 Class Diagram 39
Figure 6.1 Gantt Chart of Project Schedule. 54
Figure 8.1 DFD Level 0. 60
Figure 8.2 DFD Level 1. 61
Figure 8.3 DFD Level 2 Detection Process. 62
Figure 8.4 DFD Level 2 Alert Process.....62
Figure 8.5 ER Diagram. 63
Figure 8.6 Database field design.....64
Figure 8.7 Login Page. 65
Figure 8.8 Dashboard Interface.....65
Figure 8.9 Detection Screen. 66
Figure 8.10 History Screen Interface. 66
Figure 8.11 User Screen Interface. 67
Figure 8.12 Alerts Screen Interface. 67
Figure 8.13 Report Screen Interface. 68
Figure 8.14 Settings Screen Interface 68

List of Tables

Table 3.1 User Requirements 15
Table 3.2 Functional Requirements. 22
Table 3.3 Non-Functional Requirements. 23
Table 4.1 CRC of User. 35
Table 4.2 CRC of Detection. 35
Table 4.3 CRC of AlertRule. 36
Table 4.4 CRC of Alert. 36
Table 4.5 CRC of Report. 37
Table 4.6 CRC of ActivityLog. 37
Table 4.7 CRC of AppSetting. 38
Table 5.1 Risk Identification. 41
Table 5.2 Risk Analysis. 42
Table 5.3 Risk Planning. 42
Table 5.4 RMMM Planning for Schedule Risk. 43
Table 5.5 RMMM Planning for Technical Risk. 43
Table 5.6 RMMM Planning for Data Loss Risk. 44
Table 5.7 RMMM Planning for Security Risk. 45
Table 5.8 RMMM Planning for Performance Risk. 45
Table 5.9 RMMM Planning for Usability Risk. 46
Table 5.10 RMMM Planning for Operational Risk. 46
Table 6.1 Functionality, Input, and Output. 49
Table 6.2 Complexity Weights for External Inputs 50
Table 6.3 Complexity Weights for External Outputs 50
Table 6.4 UFP of External Inputs 51
Table 6.5 UFP of External Outputs 51
Table 6.6 UFP of External Inquiries 51
Table 6.7 UFP Contribution: Internal Logical Files 52

Table 6.8 UFP Contribution: External Interface Files 52
Table 6.9 UFP Total Summary 52
Table 6.10 TDI General System Characteristics..... 52
Table 7.1 Working Days Calculation..... 56
Table 7.2 Personnel Cost Estimation. 57
Table 7.3 Hardware Cost. 57
Table 7.4 Software Cost..... 57
Table 7.5 Other Cost. 58
Table 7.6 Accounts Table. 58
Table 9.1 Test Case: User Login..... 70
Table 9.1.a Test Case Steps: User Login. 71
Table 9.2 Test Case: Run Mock Detection. 71
Table 9.2.a Test Case Steps: Run Mock Detection..... 72
Table 9.3 Test Case: History Filtering and Export.. 72
Table 9.3.a Test Case Steps: History Filtering and Export.. 73
Table 9.4 Test Case: Alert Rule and Trigger.. 74
Table 9.4.a Test Case Steps: Alert Rule and Trigger..... 74
Table 9.5 Test Case: Report Generation.. 75
Table 9.5.a Test Case Steps: Report Generation..... 76
Table 9.6 Test Case: User Management (Admin).. 76
Table 9.6.a Test Case Steps: User Management (Admin).. 77

Chapter 1.

Introduction

1.1 Introduction

One of the most challenging everyday tasks of small and medium-sized recycling entrepreneurs in Bangladesh is solid waste handling. Waste is delivered in a mixed stream, is segregated by hand, and the quantity of waste received at a site by day of the category of waste is typically recorded on paper or in a spreadsheet that is not accessed at the end of the shift. This practicum was done to look at the recordkeeping and supervision aspect of that problem and implement it with an application that an operator can use offline, a supervisor can audit, and an administrator can set up for the site's waste categories. The application has been named the Smart Waste Management System (SWMS) and is the subject of the rest of this report. This is a modified shell, with a stubbed detection engine, for use on a desktop. The detection engine generates realistic results by adding a training model of YOLOv8 in the following stages of the project.

1.2 Background of the Study

The Department of Environment estimates that Bangladesh produces over two thousand tons of MSW daily, of which a significant proportion is collected by small private operators who sort the waste at their own premises and sell it to industrial recyclers. Typically, these operators do not have digital tooling. The supervisor will be walking through the sorting area with a notebook in hand, and will record the approximate number of waste bins containing each of the waste categories, then report this to the Office at the end of the day. This presents three operational issues that frequently arose in my discussions with the people I talked to when I was scoping out the practicum. The first one is delayed visibility. The number of plastic items or e-waste is recorded in spreadsheets at the end of the shift, meaning the supervisors only find out about an unusual amount of plastic waste or e-waste several hours after the incident. This time, the Bins are emptied, and the evidence is gone. The second problem is that an audit trail is lacking. If a customer challenges the weight or the category of a delivered batch, there's no simple way for the operator to get the record, as the paper log is stored in chronological order, not by customer or category. The third problem is alerting. Bins are not emptied, or even partially emptied, and only when there is a smell and/or the spillage is visible. No rule-based system exists that says, "if you have more than 30 plastic detections in a day, send an email to the supervisor." Similar gaps in waste management informatics are identified

in the literature, as summarized by Rahman et al. (2022) and by the World Bank's Waste 2.0 dataset, and the same low-cost approach towards creating digital records of waste, with structured alerts, is recommended as a starting point in the same way. Those three problems were the basis for the Smart Waste Management System to be scoped. The system has been designed as a desktop form factor to run on the low-cost Windows computers, which are prevalent in the office back room in Dhaka. The system is not reliant on the internet, as the local SQLite database is used. The development of a place-holding detection engine was created first, since the remaining components of the application – the application's screens, the alert engine, the report generator, and the user management-can be developed, tested, and demonstrated before the model. With the model ready, only one file in the codebase changes.

1.3 Methodology

Data for this practicum were collected from two types of data sources. They are detailed below.

1.3.1 Primary Sources.

The following methods were used to gather primary information:

a. Weekly meetings with the practicum supervisor for the purpose of agreeing on the scope of each phase and reviewing deliverables with the build document.
b. Informal discussion with the manager of one of the recycling collection points in Uttara on his daily operations, the types of waste that are usually collected, and the reasons for using paper logs.
c. Two observations on the sorting process on site, to verify the role hierarchy (administrator/ supervisor/ operator) and separation of duties between the different roles.
d. Testing of all screens and all workflows was performed hands-on during development, recording the notes in a development journal, which was reviewed at the end of each phase.

1.3.2 Secondary Sources.

The following were used as secondary sources of information:

a. Waste classification based on deep learning has been studied in academic papers, such as Yang and Thung (2017) for the TrashNet dataset, and more

recent research for the detection of waste using the YOLO deep learning framework (Adedeji and Wang, 2019).

b. Government and inter-agency data, such as the World Bank What a Waste 2.0 dataset, and the Department of Environment 2021 status report.

c. Official documentation for PyQt5, SQLAlchemy, ReportLab, and Matplotlib, used to guide implementation choices.

d. Public blog posts about bcrypt password hashing and about PyQt threading patterns.

1.4 Objectives

The goals of the proposed system are divided into a single general objective and a list of specific objectives that are derived from the general objective and provide the specific measurable deliverables.

1.4.1 Broad Objective

The general goal of the practicum is to create a stable desktop application that will record, classify, and audit the events of waste detection in a small recycling facility, will generate periodic PDF reports for the management to review, and will provide automatic notifications when waste is detected, exceeding the thresholds set by the management, where a clean interface will be provided.

1.4.2 Specific Objective.

Below are the specific goals of the project:

a. Create a relational schema in SQLite using SQLAlchemy ORM to store users, detections, alert rules, alerts, activity logs, reports, and application settings.

b. Create three roles (administrator, supervisor, operator) and define role rules on all screens of the application.

c. Create a prototype detection engine to generate plausible category names, confidence values, bin-filling values, and to store images with annotations.

d. Create a dashboard, detection screen, history screen (filter and export), alerts screen, reports screen, users screen for admin, and settings screen (SMTP settings).

e. Create PDF reports using ReportLab with summary statistics, matplotlib plots and a paginated detections table.

f. Offer an alert engine that works with the configured thresholds to send email notifications via the configured SMTP server from the detection log.

1.5 Process Model

The iterative-incremental process model was chosen for the practicum. There's enough to the system that if we had to do the waterfall run, we wouldn't risk delaying any integration till the end, and it's small enough that we'd be better off using a separate team in a heavyweight Scrum. The iterative-incremental model is a suitable model because the supervisor can attempt to judge the working increment at the end of each phase before moving on to the next. The phases used: Foundation, Core Logic, UI Widgets, UI Screens, Entry Point, Integration, and Polish (As described in the build document). The process of each iteration was the same, using the same internal review cycle of analysis, design, implementation, and verification, with a review at the interface between iterations.

Below are reasons why this model was chosen:

a. It brings integration issues to the forefront with each phase yielding a working system, and the database, the engines, and the UI are all tested at the same time from an early stage of the project.
b. It fits scope changes within a phase without disrupting the plan, which is important in a student project, where scope is prone to drift with new ideas coming up.
c. It provides an almost functional application at the end of the first two stages, so that the supervisor has something to examine well before the project is finished.
d. It can be integrated in a deferred way without being a hassle; it is a full increment that meets all the upstream expectations, so the model integration can be planned and resourced as an independent iteration without rework.
e. It fits easily into the build document, which is organized as a list of steps, in order, and including explicit verification steps.

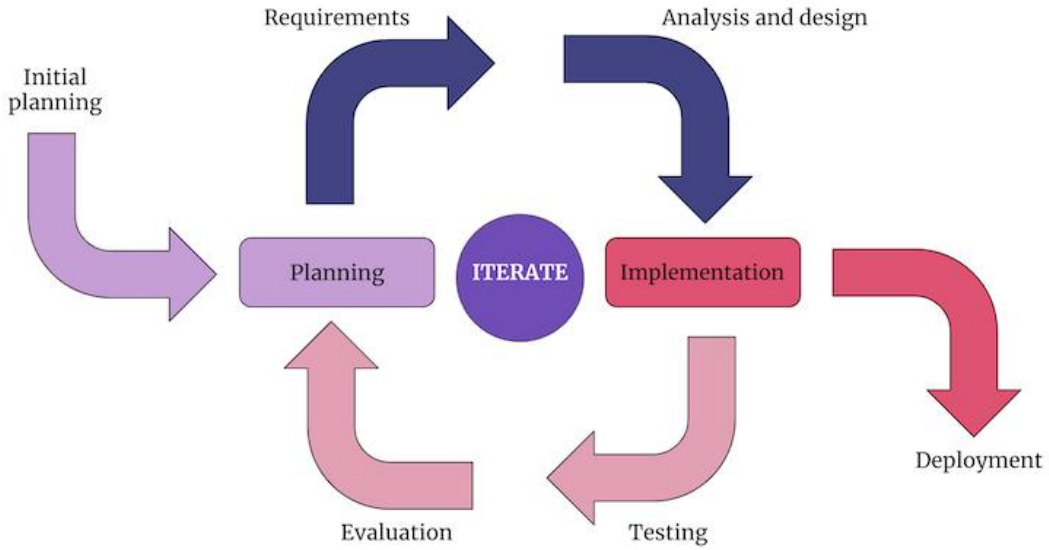

Figure 1.1 Iterative-Incremental Process Model

The cycle is shown in Figure 1.1. The four activities are repeated for each phase, with the output of one phase being the input to the next.

1.6 Feasibility Study

Before the beginning of development, three feasibility dimensions were taken into consideration. Below is a summary of the conclusions for each dimension:

1.6.1 Technical Feasibility.

We have chosen all open source, well-documented, and supported technologies for Windows 10 and Windows 11. For over a decade, PyQt5 has been the choice of a cross-platform desktop GUI framework for Python. bcrypt, Matplotlib, and ReportLab are stable Python packages with large user bases, and the default zero-configuration desktop is SQLite.SQLAlchemy, the most popular ORM in the Python world, and SQLite, the default zero-configuration database for desktops, have stable Python bindings, while ReportLab has a large user base. The placeholder detection engine is implemented without Pillow; however, it is available on most Python distributions. The hardware requirement (a Windows machine with 4 GB RAM) is similar to office machines already being used at the target facility. No technical barriers are preventing the deployment, as all components used do not require a paid license or a network connection during normal use.

1.6.2 Economic Feasibility.

Personnel time is the predominant cost in the project. The hardware was pre-existing, the software is open source, and only the cost for SMTP service outbound alerts is needed, which can be the supervisor's pre-existing Gmail account for free. The detailed breakdown is given in Chapter 7. The figures indicate that the development cost is within the budget of the host organization to maintain a system that can be used in place of paper logs and to reduce the amount of time the supervisor spends each week preparing a summary by hand.

1.6.3 Operational Feasibility.

The same role hierarchy that currently exists at the facility is used in the application. Operators still make detection-time entries, supervisors still verify and approve, and administrators still manage users and settings. The interface employs Bengali-friendly Latin characters and a sidebar mode that is similar to that of familiar business applications, which makes it easy to pick up. A brief user guide and a session-based training program are enough to implement the system, and the rollback plan in case of failure is to go back to the current paper-based logs without losing any data. No operational dependency that will inhibit adoption.

1.7 Structure of the Report

The report is organized into eleven chapters. Chapter 1 is this introduction. Fully describing the host organization is the subject of Chapter 2. The requirement engineering technique and its elicitation, analysis, specification, and the use case diagram are covered in Chapter 3. The analysis models covered in Chapter 4 are activity diagrams, swimlane diagrams, sequence diagrams, CRC cards, and the class diagram. The risk management plan and RMMM tables are described in Chapter 5. The function point estimation and project schedule are provided in Chapter 6. Estimates of the cost are found in Chapter 7. The system design is covered in Chapter 8, which includes DFD, ER Diagram, Database Field Design, and the interface designs. The test cases are given in Chapter 9. Ethics and Sustainability are discussed in Chapter 10. The report ends in Chapter 11 and then enumerates the limitations and the future works. References and plagiarism report are provided below.

Chapter 2.
Organizational Overview

2.1 Organization Overview

This chapter introduces the host organization for the practicum, describes its vision and mission, lists the services it provides, presents its organizational structure, identifies my position during the practicum period, and gives the address of its office.

The practicum was carried out at AIQNIC, a Dhaka-based company that delivers end-to-end artificial intelligence and data science solutions to businesses seeking to harness the power of AI. The company's service portfolio includes predictive analytics, natural language processing (NLP), computer vision, and MLOps. The choice of host was driven by the supervisor's existing relationship with the company and by the company's interest in applying AI-driven approaches to real-world business problems.

2.2 Organization Vision

The vision of AIQNIC is captured in its tagline: "Engineering the Future of Intelligence." The company envisions a future in which intelligent systems are accessible, reliable, and transformative for businesses of every scale.

2.3 Organization Mission

The mission of AIQNIC is to deliver end-to-end AI and data science solutions that transform the way businesses operate.

2.4 Organization Services

The services offered by AIQNIC are listed below:

a. Software Engineering – Strategic software development for business processes to identify high-impact opportunities for AI adoption, including feasibility studies and roadmap planning.

b. Predictive Analytics – Design and development of statistical and machine learning models for demand forecasting, risk scoring, customer churn prediction, and other forward-looking business applications.

c. Natural Language Processing (NLP) – Building text and speech processing solutions such as chatbots, sentiment analysis engines, document classification systems, and language translation tools.

d. Computer Vision – Development of image and video analysis applications, including object detection, facial recognition, optical character recognition (OCR), and quality inspection systems.

e. MLOps and Model Deployment – End-to-end machine learning operations including model training pipelines, continuous integration and delivery for ML models, monitoring, retraining, and production infrastructure management.

f. Data Science Solutions – Comprehensive data engineering, data analysis, and visualization services that help organizations structure, clean, and derive actionable insights from their data assets.

2.5 Organizational Structure

AIQNIC operates with a lean, project-driven organizational structure suited to a technology consulting firm. The Chief Executive Officer (CEO) sits at the top of the hierarchy. Reporting to the CEO are the heads of the core functional areas: the AI and Engineering Division, the Business Development Division, and the Administration Division. The AI and Engineering Division is led by a technical lead and comprises AI/ML engineers, data scientists, and software developers who work on client projects. The Business Development Division handles client acquisition, partnerships, and market outreach. The Administration Division oversees human resources, finance, and office operations. The structure is shown in Figure 2.1.

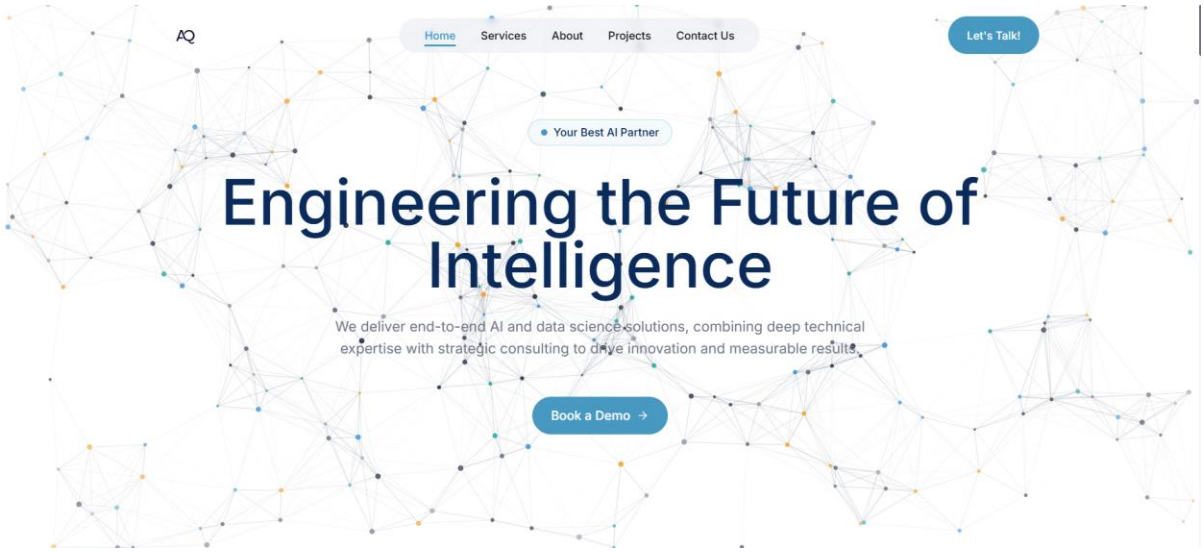

Figure 2.1 Organizational Structure

2.6 My Position in this Organization

I joined the AI and Engineering team at AIQNIC as a Jr. Artificial Intelligence Engineer Intern for the practicum period. My role was to design and develop software solutions as assigned, working alongside AI/ML engineers and data scientists. I reported daily to the team lead and weekly to my practicum supervisor at IUBAT. The role gave me direct exposure to the company's project workflow, access to real datasets for development and testing, and hands-on experience with the tools and technologies used in production AI projects.

2.7 Address of the Organization

House-177, Road-11, Block-B, Jahangirnagar Co-operative Housing Society, Savar, Dhaka-1343, Bangladesh

Chapter 3.

Requirement Engineering

3.1 Requirement Engineering

The job of requirement engineering is to get to know the system's needs, to document them in a format that can be used in system design, and to validate them with the people whose problems are being addressed. This chapter provides a brief overview of the four steps that led to the creation of SWMS, the methods of elicitation used to feed the inputs, the analysis used to group the raw input into themes, and the specification used to convert the themes into testable statements. The chapter ends with the use case diagram for the system.

The requirement engineering activity of SWMS was performed in a sequence of three steps. For every step, an artefact was showcased to the practicum supervisor prior to the next step.

a. Elicitation: gather unprocessed information from the host organization, supervision, and literature review.
b. Analysis: Group the raw input into user, system, functional, and non-functional, and resolve any duplications and flag any conflicts.
c. Specification: list each requirement as a numbered testable statement; record the use case diagram.

3.2 Requirement Elicitation

The process of elicitation started with an unstructured interview with the host organization. The IT team lead and operations supervisor attended the session. The goal was to document the current process, the problems being faced, and the hopes they held for a digital solution to paper logs. This session generated approximately two pages of notes related to authentication, recordkeeping, alerts, and reporting. A second session was conducted a week later to confirm the recognition of the role hierarchy and discuss the procedure for confirming recognition of the detection results that the supervisor wished to keep manual for the first deployment. At the same time, the literature review brought up the typical non-functional issues of desktop applications in industrial environments; that of response time, password handling, error recovery, and offline operation. These were put on the elision backlog. Lastly, the practicum supervisor saw the backlog and suggested two more points from the project's academic side of the project: the need for CSV and Excel export for the data, and the

organization of the codebase to be used for integration of the model in the next phase of the project.

3.3 Requirement Analysis

Themes were grouped from the raw input collected in the elicitation step as follows.

1. Security measures for authentication and access control. The system must recognize at each action who the operator is, must apply different permission sets to the three roles, and must ensure the security of the passwords at rest.
2. Detection and recordkeeping. Each detection event needs to be stored with the time of the detection, the operator that performed the detection, the resulting image, the categories detected, the confidence level, and the bin fullness indicator.
3. Verification. Supervisors will check pending detections and either approve or reject and can include comments. The verification status forms a part of the audit trail.
4. Alerts and notifications. Administrators set up rules using thresholds and periods. The system checks the rules after each detection and notifies by e-mail when it detects a violation of the rules.
5. Analytics and reporting. The Dashboard provides an overview of what has happened today. The report engine creates PDF reports on a daily, weekly, monthly, and custom range.
6. User management. Administrators add and edit operators and supervisors, and disable them. Self-deactivation isn't possible.
7. Settings and audit. These can be set as SMTP, company name, and detection threshold. All critical actions add to an activity log, which can be queried later.
8. Operational quality. Long-running tasks should not block the user interface. Errors should be reported in a clear manner. The app should function offline for all purposes (except for sending emails).

Themes were discussed with the supervisor and were validated. These were used as the headings for the specification step to follow.

3.4 Requirement Specifications

In this step, we documented detailed requirements for our system. User requirements, System requirements, Functional and Non-functional requirements are included here.

3.4.1. User Requirements

User requirements are written from the perspective of the people who use the system. They are listed in Table 3.1.

Table 3.1 User Requirements.

UR#	Description	Primary Role
UR-01	The user can log in with a username and password.	All
UR-02	The user can change their password from the settings screen.	All
UR-03	The user can upload an image and run waste detection on it.	Operator
UR-04	The user can see my own detection history with the original and annotated images.	Operator
UR-05	The user can see all detection records, filter by date, category, and status.	Supervisor, Admin
UR-06	The user can verify or reject any pending detection and add a note.	Supervisor, Admin
UR-07	The user can export the detection list as CSV or Excel for offline analysis.	Supervisor, Admin
UR-08	The user can view a dashboard with today's totals, the most common waste type, and active alerts.	All
UR-09	The user can generate a PDF report for a daily, weekly, monthly, or custom date range.	Supervisor, Admin
UR-10	The user can configure alert rules with a category, a threshold, a period, and a notification email.	Admin
UR-11	The user can acknowledge alerts, so the team knows they are being handled.	Supervisor, Admin
UR-12	The user can manage user accounts: add, edit, deactivate, and reactivate.	Admin
UR-13	The user can configure the SMTP settings for outbound email.	Admin
UR-14	The user can view the activity log to see who did what and when.	Admin
UR-15	The user can take a backup of the database file from the settings screen.	Admin

3.4.2. System Requirements

System requirements outline the modules of the application and the actions performed by each module. Here is a list of modules, ordered alphabetically and following the same order as the build document of the project.

1. Authentication and Session Handling

   1.1. User launches the app and is presented with a login screen.

   1.2. User types of their login name and password and click on the Login button.

   1.3. The stored bcrypt hash for the username is retrieved and compared against the password entered.

   1.4. If the match succeeds, the system updates the 'Last Logged In' date on that user and brings up the main app window.

   1.5. On a failed match, the system shows an “Invalid credentials” message and keeps the user on the login screen.

   1.6. The user's role (admin, supervisor, or operator) is loaded, and a check_permission helper is exposed that takes the hierarchy of admin, supervisor, and operator into account for all actions protected by the system.

   1.7. When the user clicks on Logout in the sidebar, it will terminate the current session and log the user out of the computer.

2. User Management

   2.1. Admin clicks on Users from the left side menu to open the user’s management screen.

   2.2. Admin pushes the Add User button, enters username, full name, email, role, and initial password, and clicks Save.

   2.3. The password is hashed in the system using bcrypt, the new user is added, and an entry is added to the activity log.

   2.4. Admin clicks on a user and then Edit to update the user's name, emails, or role, then Save to save the change.

   2.5. Admin chooses a user and clicks Deactivate to disable the account, which is not possible if the user selected to be deactivated is the currently logged-in admin.

   2.6. Admin decides to reactivate the user and clicks Reactivate to restore access to the user.

2.7. One entry will be added to the activity log for every create, update, deactivate, and reactivate action.

3. Detection Execution

3.1. Click Detection in the left sidebar to launch the detection screen.

3.2. The operator clicks the Upload Image button and selects an image file from the local file system.

3.3. The system shows a preview of the image that is selected and allows the Run Detection button.

3.4. The operator clicks Run Detection; the detection engine runs in a background QThread, thus not freezing the UI.

3.5. The detection engine produces between one and four detections per of the seven configured waste categories with a confidence score ranging from 0.65 to 0.98 and a bin-fill level.

3.6. The system looks at a copy of the original image and can draw a bounding box around each detected object, saving the annotated image to the results directory.

3.7. A detection record is saved to the database, with the image path, annotated image path, operator ID, list of categories, confidence scores, fill level, status (Pending), and a time stamp.

3.8. It shows the annotated image and detection summary to the operator and logs the activity to the activity log.

4. Detection History and Export

4.1. To view the detection history screen, click on History on the left-hand side of the screen.

4.2. Records are loaded into a paginated table by the system, 25 rows per page.

4.3. The operator can only view their detection records, while supervisor/admins can view all records.

4.4. User clicks on filters by date range, category, status, or operator, and then clicks Apply to update the table.

4.5. By clicking any row, the user can see the original and annotated image side by side and detailed information about the detection.

4.6. On the currently selected list, the Supervisor/Admin clicks the Export CSV or Export Excel button to export the filtered list to disk.

4.7. An export activity entry is written to the activity log.

5. Detection Verification Workflow

5.1. Supervisor/Admin clicks on History and filters to Pending status.

5.2. User clicks the option to verify the detection or reject the detection as invalid.

5.3. User optionally types a note in the verification dialog and clicks Save.

5.4. The detection record is updated with the new status, verifier ID, verification timestamp, and note.

5.5. The system will record a verification record in the activity log.

6. Analytics and Dashboard

6.1. Click on Dashboard from the left-hand nav panel to view the analytics screen.

6.2. The system calculates and shows the total detections for today, the total detections in the system so far, the most frequent waste type, and the number of active alerts.

6.3. The system calculates and shows the distribution of categories and the distribution of the level of filling for the selected period.

6.4. The system calculates and displays the number of detections collected each day as a chart for the period of interest.

6.5. The system calculates and reports on operator performance metrics such as detections/operator and verification rate.

6.6. User updates the time period filter and changes all metrics and charts on the dashboard.

7. Report Generation

7.1. Supervisor/Admin clicks Reports on the left-hand side to display the Reports screen.

7.2. User chooses a period of time to generate a report (daily, weekly, monthly, or custom date range) and clicks Generate Report.

7.3. The report engine fetches and processes the detections for the selected period and executes in a QThread.

7.4. The system produces a category distribution plot using Matplotlib and includes it in a PDF.

7.5. The PDF contains summary statistics, a fill-level summary, and a paginated detection table for the defined time range.

7.6. The system saves the PDF in the reports folder and adds the file path, period, and generator ID to the reports table.

7.7. A report generation entry is written to the activity log, and the user clicks Open to display the report.

8. Alert Rule Management

8.1. Admin clicks on Alert Rules found on the left-hand side to view alert rules screen.

8.2. Admin enters Category, Threshold count, Period (hourly, daily, weekly), Notification e-mail, and active flag, and clicks Save.

8.3. The system adds the rule and records an addition to the activity log.

8.4. Admin clicks on the rule from the list and clicks Edit to edit any field, and then clicks Save.

8.5. Admin chooses a rule and clicks Delete to delete it; the system prompts for confirmation.

8.6. Admin switches the active flag for a rule on and off to enable or disable it, without deleting it.

9. Informal Assessment and Record of Learning.

9.1. Once a successful detection is made, the alert engine looks at all active rules and compares them to the detection counts for the rule's category and period as recorded.

9.2. If the threshold of the rule is exceeded, a new alert row is added to the system with the rule ID, the category, the number of breaches, the breach timestamp, and the status (unacknowledged).

9.3. The system forwards the alert to the notification service for email delivery if the notification email is set in the rule.

9.4. Supervisor/ Admin clicks Alerts in the left sidebar to see the list of alerts with their status.

9.5. User clicks on an unacknowledged alert and Acknowledge to mark the alert as handled.

9.6. A list of the acknower ID, the time, and date is recorded, and an activity log entry is written.

10. Email Notification Service
    10.1. The notification service gets SMTP host, port, username, password, and sender address from the application settings table.
    10.2. Using the rule details, breach count, and time, along with an alert payload received from the alert engine, the service formats an email message.
    10.3. The service sends out the e-mail via smtplib over a secure connection.
    10.4. The service notifies the alert engine of success or failure.
    10.5. Failure: The system will not block the detection workflow and will log the failure reason in the activity log.
11. Activity Logging
    11.1. Activity log entries are made for the following events: Login, Detection Run, Detection Verification, User Create/Edit/Deactivate/Reactivate, Alert Rule Create/Update/Delete, Alert Acknowledgment, Report Generation, and any setting changes.
    11.2. Each entry contains the user ID of the actor, type of action, target entity, time the action occurred, and a (facultative) description.
    11.3. Admin clicks on Activity Log on the left sidebar and is led to a paginated table displaying the list of all log entries at 25 per page.
    11.4. Admin can use the "filter" feature to restrict the log view, based on a date range, action type, or user.
12. Application Settings
    12.1. Admin clicks on Settings in the left sidebar, which brings up the settings screen.
    12.2. Admin enters or modifies the SMTP host, port, username, password, and sender address and presses Save.
    12.3. The company name and default confidence threshold are entered or changed by Admin, and then the Save is clicked.
    12.4. All settings are saved as key-value pairs in the application settings table, and an entry is added to the activity log.
    12.5. The system takes the SQLite database file to a user-specified location and displays a toast message indicating success or failure.
13. User Interface Shell
    13.1. It comes with a left sidebar and a stacked content widget on the right.

13.2. The navigation buttons (Dashboard, Detection, History, Reports, Alerts, Alert Rules, Users, Activity Log, Settings) are shown in the sidebar, based on the user's role.

13.3. The buttons in the sidebar switch the right-hand stacked widget to the corresponding screen.

13.4. At the top right corner of the window, toast notifications will appear on the shell for success, warning, and error events.

13.5. A loading spinner appears over the shell when it is running an action that takes a long time to complete, like a detection or report, or a database backup.

13.6. At the bottom of the sidebar, a Logout button can be found on the shell, which will log out of the session and return the user to the logon screen.

14. Concurrency

14.1. All access to the database and to the file system is through a QThread, which is not the UI thread.

14.2. UI thread is responsive while detecting, Report generation, CSV / Excel, E-mail sending, and database backup.

14.3. The background threads send the signal when done, the UI thread receives the signal, and updates the appropriate screen.

14.4. In case of thread failure, an error toast appears on the UI thread, and results are recorded in the activity log.

15. Persistence

15.1. The first time it is run, it will generate a SQLite database file in the database/waste_management.db file.

15.2. The system generates all the necessary tables, including users, detection, report, alert, alert_rules, activity_log and settings.

15.3. The system automatically generates a default administrator account with a known initial password that should be changed once you log on as the admin for the first time.

15.4. The system default settings for applications (SMTP placeholders, company name, default confidence threshold).

15.5. Each write operation in the database is part of a transaction; failures are rolled back and reported as toast notifications.

3.4.3. Functional Requirements

Functional requirements describe what a system should do. They explain the features that users can use, like login, register, logout, package activation, and generating reports. The functional requirements are listed in Table 3.2 with one row per requirement.

Table 3.2. Functional Requirements.

FR#	Description	Priority
FR-01	The system requires a username and a password to grant access.	High
FR-02	The system shall hash passwords with bcrypt and store only the hash.	High
FR-03	The system shall update last_login on every successful authentication.	Medium
FR-04	The system allows an operator to upload an image of file type PNG, JPG, or JPEG and run a detection on it.	High
FR-05	The system shall save every detection event with image path, result image path, waste category, confidence, fill level, operator, status, and timestamp.	High
FR-06	The system shall allow supervisors and administrators to verify or reject any pending detection.	High
FR-07	The system shall allow supervisors and administrators to filter detections by date range, category, and status, and to export the filtered list.	High
FR-08	The system shall display a dashboard with today's detection count, the most common category, and the active alert count.	Medium
FR-09	The system should render a category distribution chart and a daily-count chart for the configured period.	Medium
FR-10	The system shall generate a PDF report for daily, weekly, monthly, or custom date ranges.	High
FR-11	The system shall allow administrators to define alert rules with category, threshold, period, and notification email.	High
FR-12	The system shall evaluate active alert rules after each detection and create alert rows when thresholds are breached.	High
FR-13	The system shall send an email through the configured SMTP server when a rule with a notification email is breached.	Medium
FR-14	The system shall allow supervisors and administrators to acknowledge alerts.	Medium
FR-15	The system shall allow administrators to add, edit, deactivate, and reactivate users, but never to deactivate themselves.	High
FR-16	The system shall write an activity log entry for every privileged action.	Medium

3.4.4. Non-Functional Requirements

Non-functional requirements describe how a system should work. They focus on qualities such as performance, security, reliability, usability, and scalability. Non-functional requirements describe how the system has to behave rather than what it has to do. They are listed in Table 3.3.

Table 3.3. Non-Functional Requirements.

Category	Requirement	Target
Performance	A mock detection on a 1080p image shall complete within 2 seconds on a 4 GB RAM Windows laptop.	< 2 s
Performance	A PDF report covering one month with up to 1,000 detections shall be generated within 10 seconds.	< 10 s
Usability	An operator shall be able to run a detection from a cold start in under five clicks.	5 clicks
Usability	All destructive actions require an explicit confirmation dialog.	Required
Reliability	A failure during PDF generation shall leave the database in a consistent state.	ACID
Security	Passwords shall never be transmitted or stored in plain text.	bcrypt
Security	SMTP credentials shall be stored in the application settings table and never written to logs.	Required
Maintainability	The mock detection engine shall expose the same method signature as the planned model engine, so the model integration only changes one file.	Required
Maintainability	The codebase shall keep core, database, and UI layers in separate Python packages.	Required
Portability	The application shall run on Windows 10 and Windows 11 without modification.	Required
Offline	The application should function without internet access for every action except sending email.	Required
Auditability	Every privileged action shall be retrievable from the activity log within 30 days.	Required

3.5 Use Case Diagram

In this case, the use case diagram of the SWMS is presented in Figure 3.1 below. The system is operated by three actors: the Administrator, the Supervisor, and the Operator. The use cases are presented in such a way that the inheritance of permission is reflected: all the

cases from the supervisor are also included in the case of the administrator; all the cases from the operator are included in the case of the administrator and the supervisor.

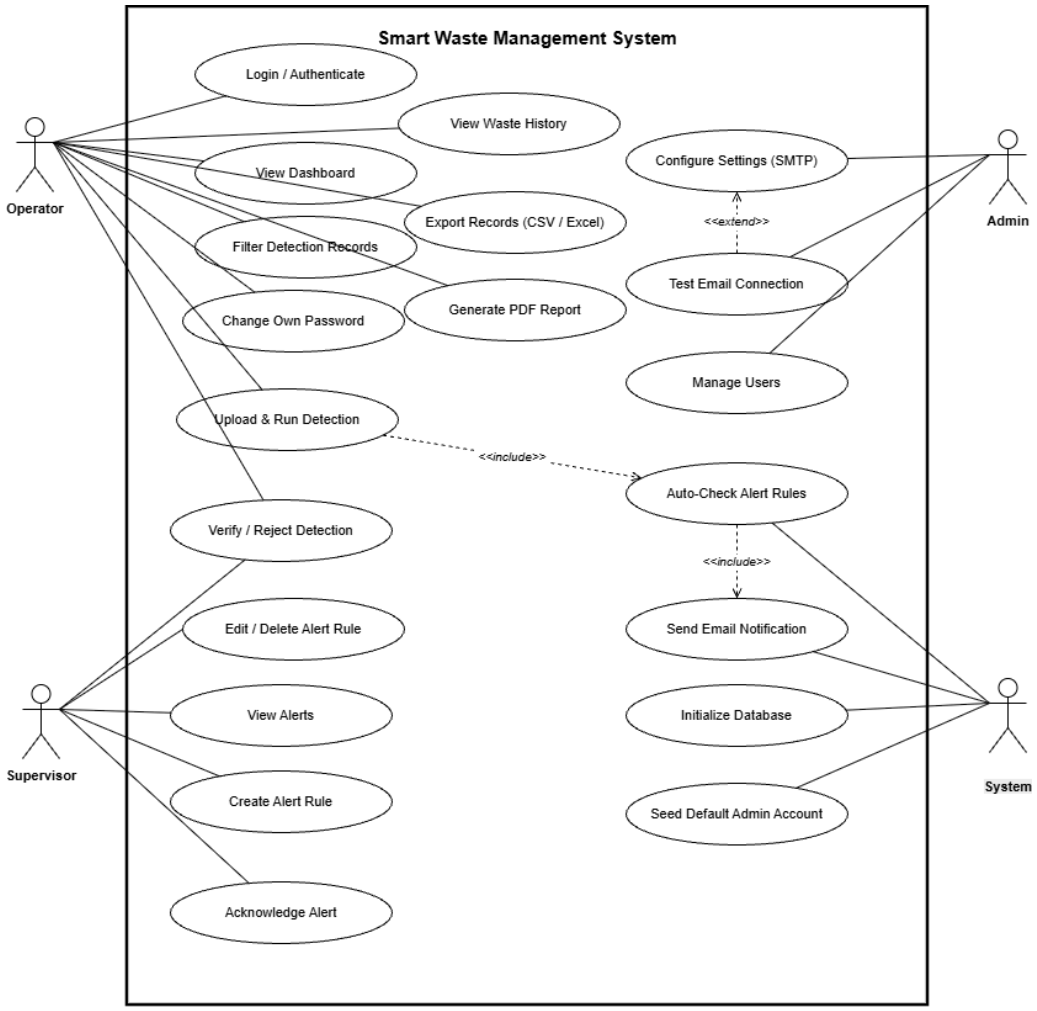

Figure 3.1 Use Case Diagram of SWMS.

It is easy to see from the diagram that the operator has a small surface, which includes login, running detection, and displaying their own history. The supervisor has a surface with added verification and reporting, while the administrator has a surface with added user and settings management. All the role-based checks that were displayed on different screens were derived from the diagram, and any difference that arose during testing was corrected by making sure that the code matched the diagram.

Chapter 4.
Analysis Modeling

Analysis modeling transforms the requirements from a list of statements to a set of diagrams that represent the dynamic and static structure of the system. It includes four diagrams in SWMS: Activity diagrams, Swimlane diagrams, Sequence diagrams, and CRC cards. The chapter ends with the class diagram that summarizes the static structure.

4.1 Activity Diagram

The four most significant use cases of the system with activity diagrams were drawn. Decision points are represented by diamonds, and each diagram begins with a solid black circle and ends with a circle around a solid circle. The diagrams are presented individually in the figures below.

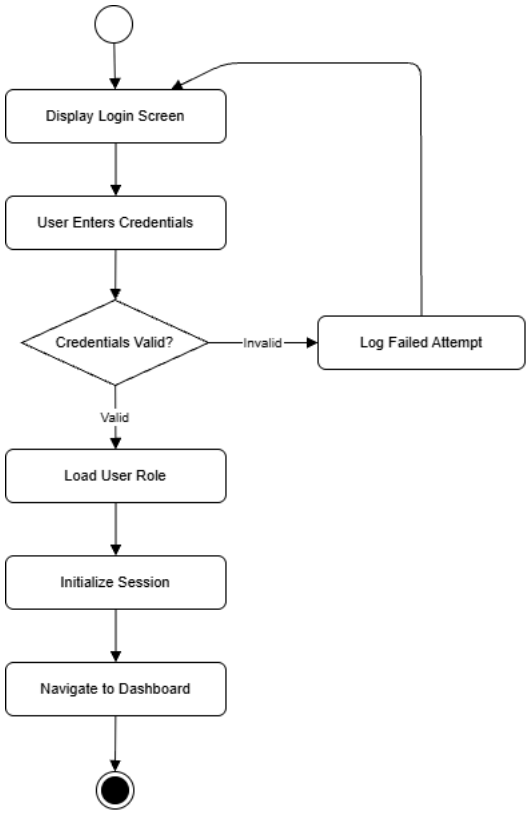

Figure 4.1 Activity Diagram: User Authentication

The user authentication flow is illustrated in Figure 4.1. There is only one decision diamond. If the login fails, the user will not be advanced, and will be logged silently (no output) to not reveal information about the existence of usernames.

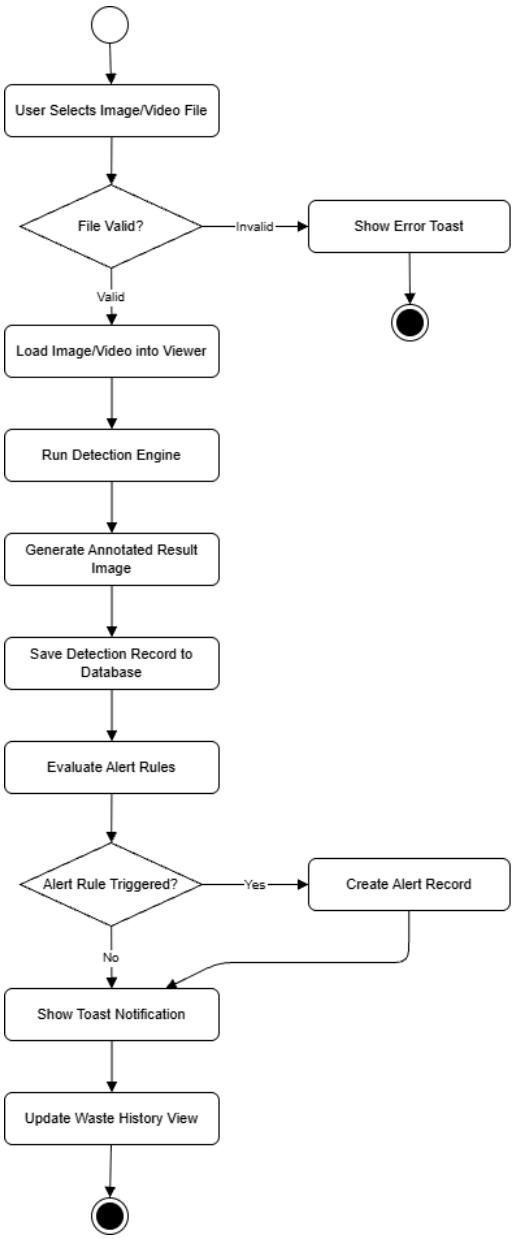

Figure 4.2 Activity Diagram: Waste Detection

The detection flow is shown in Figure 4.2. The placeholder engine is used where there is no real model, but it generates the same shape of output, and all the downstream activities (alert evaluation, toast notification, history update) will be the same when the real model is later plugged in.

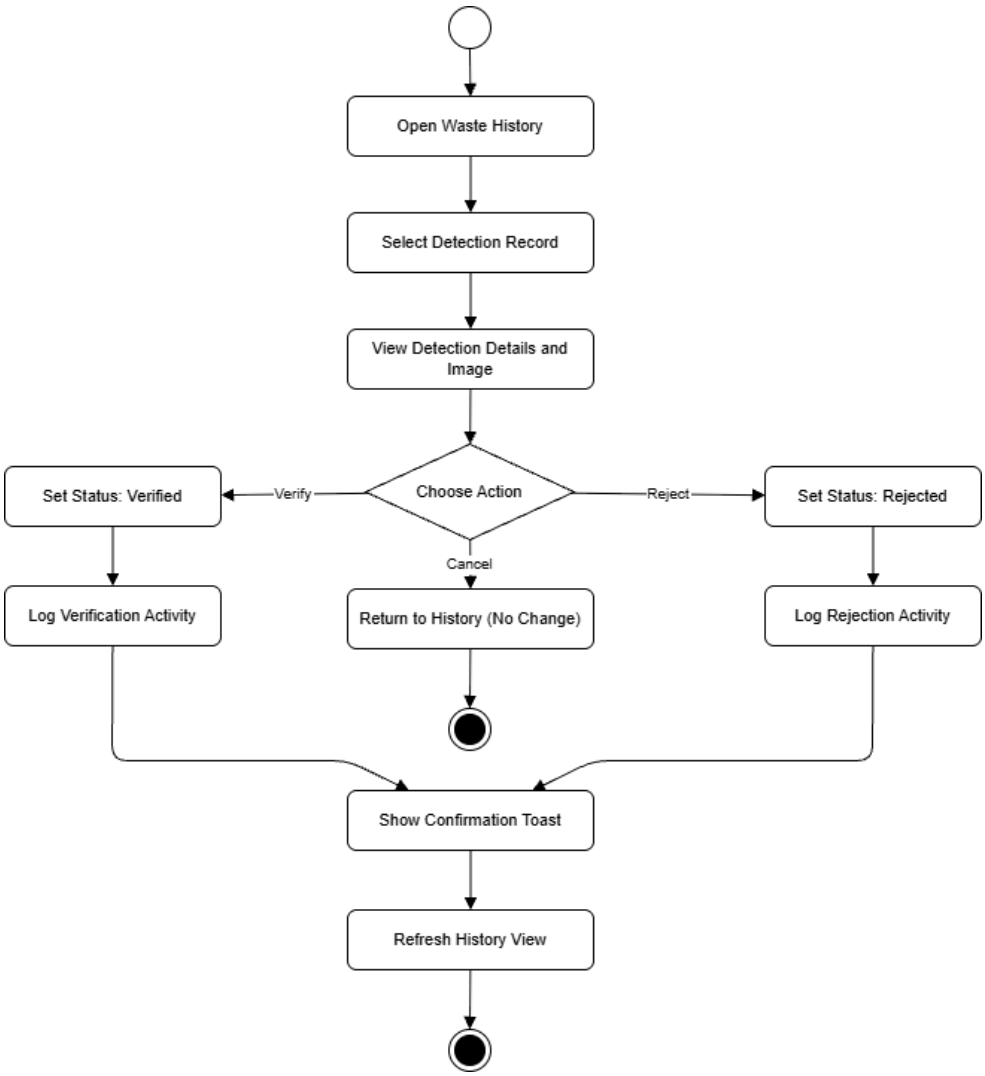

Figure 4.3 Activity Diagram: Detection Verification.

Supervisors' and administrators' verification flow is depicted in Figure 4.3. The branching in the activity is made explicit by the diagram, which is one of the few locations where the behavior of the downstream activity is dependent on the user's choice.

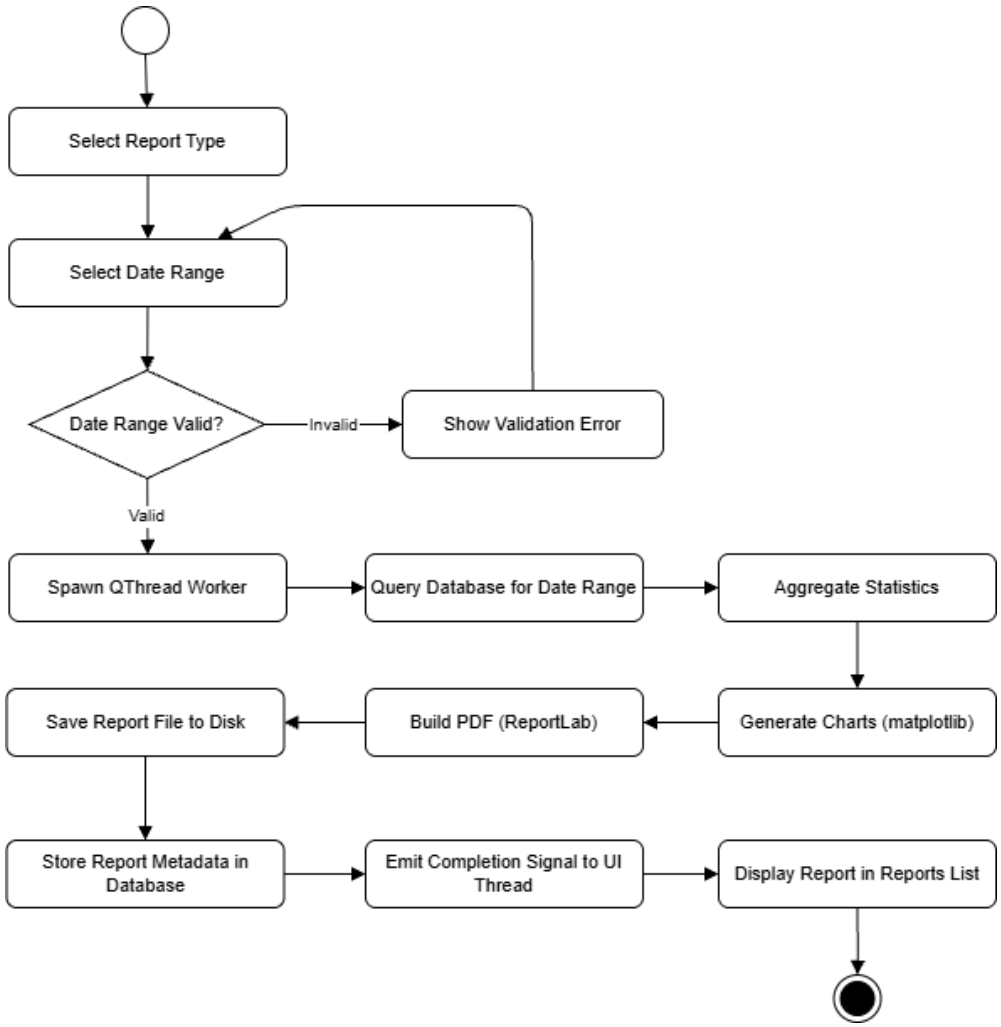

Figure 4.4 Activity Diagram: Report Generation.

The report generation flow is given in Figure 4.4. The QThread step is added as it can take a few seconds to build the report for a large date range, and the user interface would otherwise become frozen.

4.2 Swimlane Diagram

Swimlane diagrams partition activities into vertical lanes, one per actor or component. They are useful when the same activity is performed differently by different roles. Three swimlane diagrams were drawn.

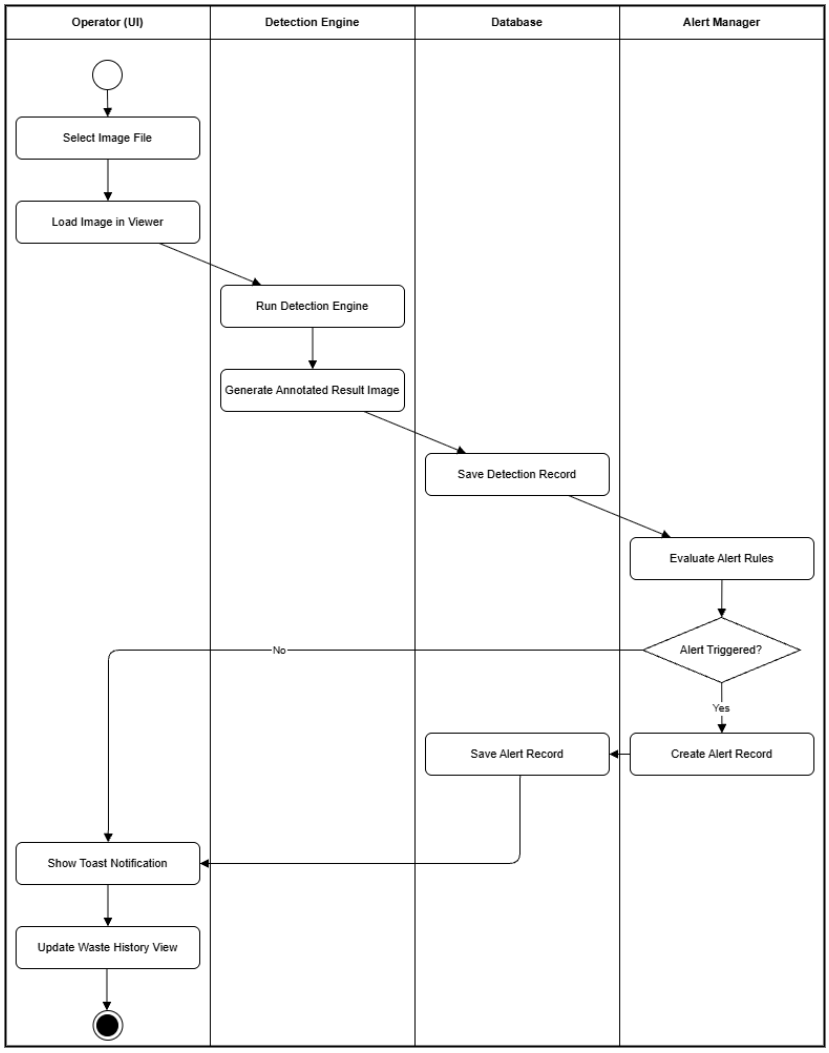

Figure 4.5 Swimlane Diagram: End-to-End Detection Workflow.

Figure 4.5 shows the operator workflow with the three downstream components rendered as separate lanes. The diagram clarifies which component owns which step and was used during code review to confirm that responsibilities were not leaking across module boundaries.

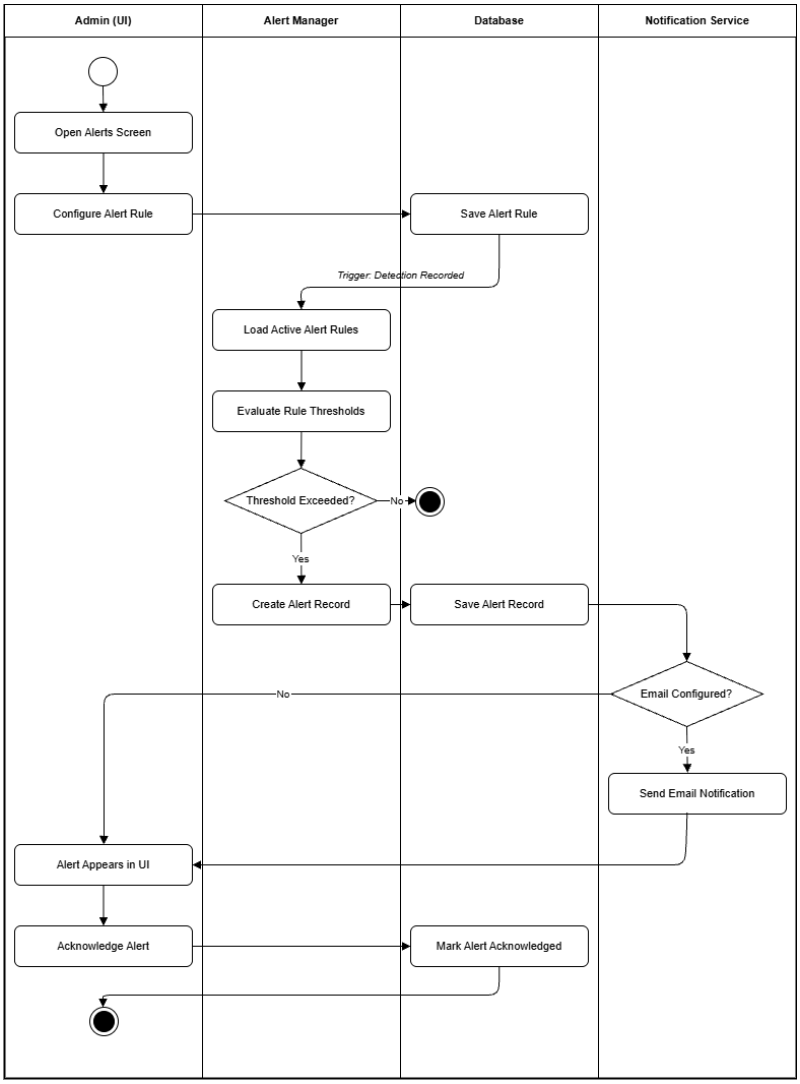

Figure 4.6 Swimlane Diagram: Alert Rule Lifecycle

Figure 4.6 shows the alert rule lifecycle. The lifecycle starts when an administrator creates the rule and continues every time a detection is recorded. The Notification Service is shown as a distinct lane because its failure mode (SMTP outage) must not break the alert flow.

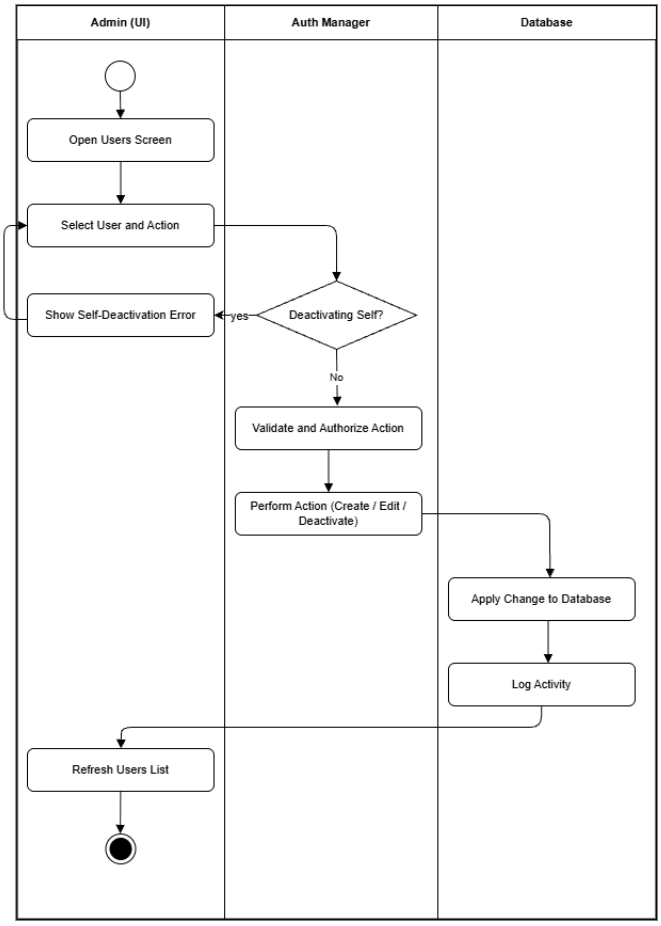

Figure 4.7 Swimlane Diagram: User Management.

Figure 4.7 shows the user management swimlane. Note the explicit guard against self-deactivation, which is a real bug class encountered in similar systems and is captured as a discrete decision step

4.3 Sequence Diagram

Sequence diagrams capture the timing of messages between collaborating objects. Two sequence diagrams were produced for the most message-rich workflows.

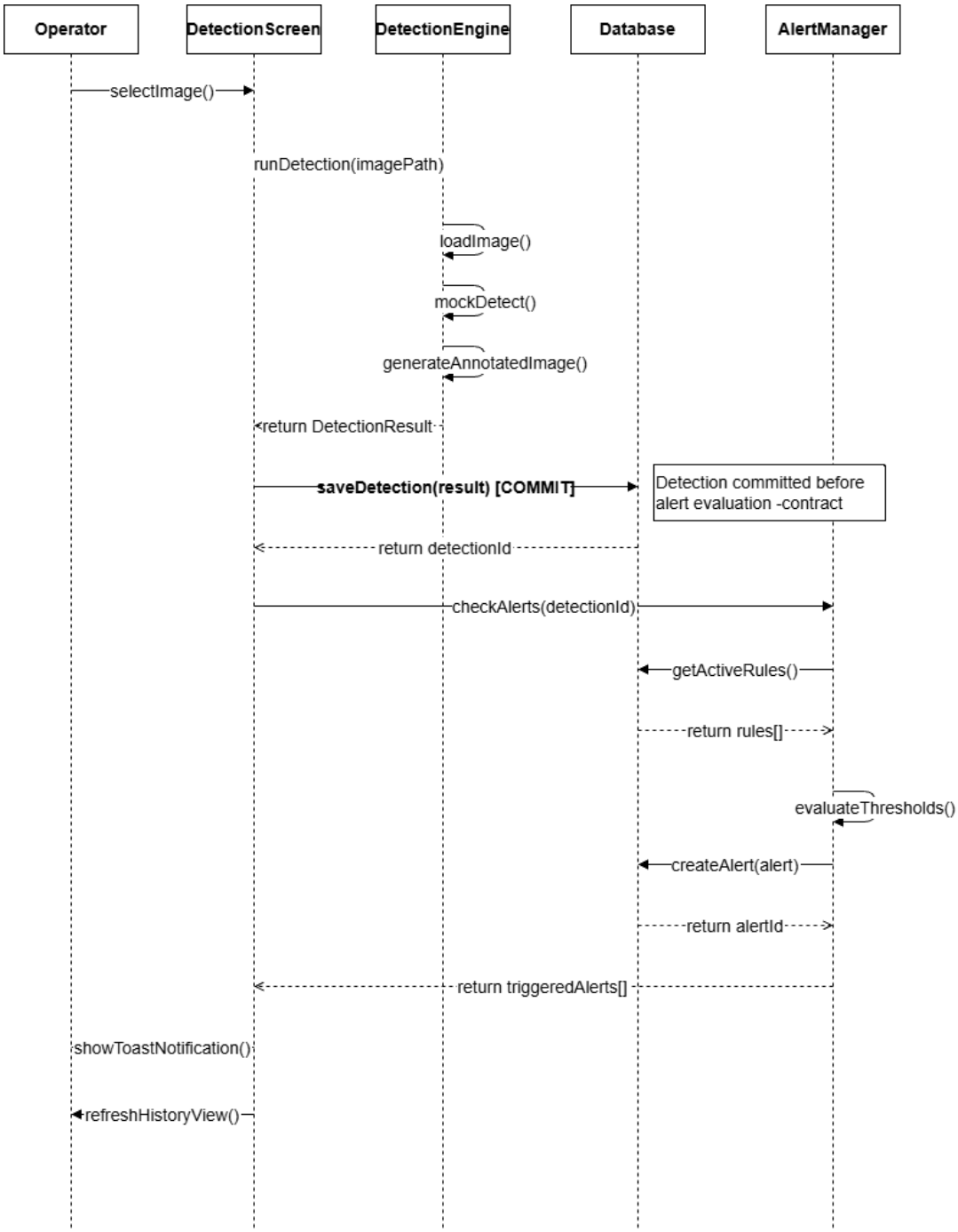

Figure 4.8 Sequence Diagram: Detection Process.

Figure 4.8 shows the messages exchanged during a detection. The diagram encodes one critical contract: the alert evaluation runs after the detection has been committed, so a partially saved detection cannot trigger a phantom alert.

Figure 4.9 Sequence Diagram: Alert Trigger Flow.

Figure 4.9 shows the alert trigger flow. The optional fragments make the conditional behavior explicit, and the SMTP error path returns False without raising, so the calling code never crashes because of a transient mail outage

4.4 CRC Cards

CRC cards (Class, Responsibility, Collaborator) are a lightweight technique for recording the responsibilities of each class and the other classes it relies on. Seven CRC cards were produced for the SWMS classes.

Table 4.1 CRC of User.

Class Name: User	
Class Type: Domain Entity	
Class Characteristics: Concrete; identifiable; persisted	
Responsibilities: • Hold the credentials, role, and activation state of a person who interacts with SWMS. • Provide a stable identifier for foreign keys in detections, alerts, reports, and activity logs. • Track last login timestamp and account creation timestamp.	**Collaborators:** • AuthManager (validates credentials and toggles active state). • Detection (records the operator and verifier). • Alert (records the acknowledger). • ActivityLog (records the actor).

Table 4.2 CRC of Detection.

Class Name: Detection	
Class Type: Domain Entity	
Class Characteristics: Concrete; identifiable; persisted; produced by the detection engine	
Responsibilities: • Hold the original image path, the annotated result image path, the predicted waste category, the confidence score, and the bin fill level.	**Collaborators:** • DetectionEngine (creator). • User (operator and verifier). • AnalyticsEngine (reads aggregate counts).

• Record who performed the detection and who later verified or rejected it. • Carry the verification status that gates downstream reporting.	• ReportEngine (lists detections in PDFs).

Table 4.3 CRC of AlertRule.

Class Name: AlertRule	
Class Type: Domain Entity	
Class Characteristics: Concrete; identifiable; persisted; configurable by administrators	
Responsibilities: • Capture the category, threshold, period, and notification email for a single alert rule. • Carry an activation flag so rules can be turned off without deletion. • Record who created the rule and when.	**Collaborators:** • AlertManager (evaluator). • Alert (children produced when the rule fires). • User (creator).

Table 4.4 CRC of Alert.

Class Name: Alert	
Class Type: Domain Entity	
Class Characteristics: Tangible, Persistent, Sequential	
Responsibilities: • Hold the message, severity, and triggered timestamp for a fired alert.	**Collaborators:** • AlertRule (parent). • AlertManager (creator).

• Track acknowledgement status and the acknowledging user. • Reference to the rule that produced it.	• NotificationService (consumer for email dispatch). • User (acknowledger).

Table 4.5 CRC of Report.

Class Name: Report	
Class Type: Structure	
Class Characteristics: Concrete; identifiable; persisted; produced by the report engine	
Responsibilities: • Capture the report type, the date range, and the file path of the generated PDF. • Record the user who generated the report and the generation timestamp.	**Collaborators:** • ReportEngine (creator). • User (generator). • AnalyticsEngine (data source).

Table 4.6 CRC of ActivityLog.

Class Name: ActivityLog	
Class Type: Domain Entity	
Class Characteristics: Concrete; identifiable; persisted; append-only	
Responsibilities: • Record a single audit event with actor, action, optional details, and timestamp.	**Collaborators:** • LogManager (writer and reader). • User (actor). • All modules that perform privileged actions (callers).

• Provide a stable history that can be queried by user, by action, and by date range.	

Table 4.7 CRC of AppSetting.

Class Name: AppSetting	
Class Type: Domain Entity	
Class Characteristics: Concrete; identifiable by key; persisted; mutable by administrators	
Responsibilities: • Store named configuration values for SMTP, company branding, and detection threshold. • Allow values to be read and written without disturbing the schema.	**Collaborators:** • SettingsScreen (editor). • NotificationService (consumer of SMTP keys). • DetectionEngine (consumer of threshold).

4.5 Class Diagram

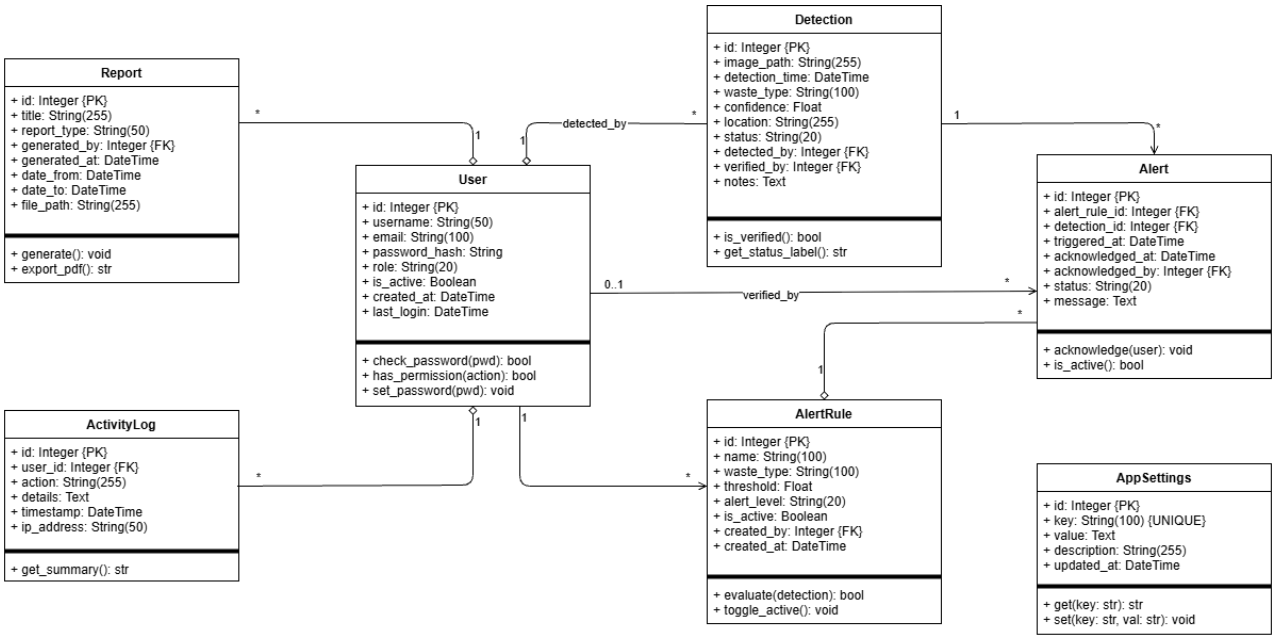

Figure 4.10 Class Diagram.

Chapter 5.
Risk Management

5.1 Risk Management

Risk management is the discipline that lets a project anticipate problems before they occur, plan a response, and react with a known playbook when a problem surfaces. The practicum followed the four standard stages of risk management: identification, analysis, planning, and an RMMM (Risk Mitigation, Monitoring, and Management) plan for each non-trivial risk. The chapter walks through each stage in turn. Risks for SWMS were grouped into the following categories. Each category contains one or more concrete risks documented in the tables that follow. Stages of Risk:

a. Risk Identification
b. Risk Analysis
c. Planning
d. Monitoring

5.2 Risk Identification

Table 5.1 Risk Identification.

ID	Risk Name	Description
R1	Schedule slip	A phase takes longer than estimated, and the project misses a milestone agreed with the supervisor.
R2	Model integration mismatch	The trained YOLOv8 model returns labels or formats that do not match the placeholder contract, requiring rework in the detection screen and the alert engine.
R3	Data loss	The SQLite database file or the result images on disk are lost due to disk failure, accidental deletion, or a corrupted shutdown.
R4	Credential exposure	Plain-text passwords or SMTP secrets leak into logs, screenshots, or version control.
R5	Performance degradation	Loading the history table or the dashboard becomes slow once the database holds tens of thousands of detections.
R6	Usability rejection	Operators find the screens too dense or the navigation too unfamiliar and revert to paper logs.
R7	SMTP outage	The configured SMTP server is unreachable, alerts queue up, and the supervisor never learns about a breach.

5.3 Risk Analysis

Table 5.2 Risk Analysis.

ID	Risk Name	Probability	Impact	Combined Severity
R1	Schedule slip	Medium	Medium	Medium - manageable with reprioritization
R2	Model integration mismatch	Medium	High	High - explicit interface contract is the mitigation
R3	Data loss	Low	High	Medium - regular backups close the gap
R4	Credential exposure	Low	Very High	High - bcrypt and key isolation reduce the probability
R5	Performance degradation	Low	Medium	Low - indexing and pagination are sufficient
R6	Usability rejection	Low	High	Medium - early demos and feedback rounds reduce the probability
R7	SMTP outage	Medium	Medium	Medium - retry logic and fallback notification absorb the risk

5.4 Risk Planning

Table 5.3 Risk Planning.

ID	Risk Name	Strategy	Headline Action
R1	Schedule slip	Mitigate	Hold the phase boundaries and trim scope inside a phase rather than extend its duration.
R2	Model integration mismatch	Mitigate	Freeze the detection engine interface in code and write unit tests that exercise it with sample inputs.
R3	Data loss	Mitigate	Provide an in-app backup action and document a daily backup routine.
R4	Credential exposure	Avoid	Use bcrypt for passwords and store SMTP secrets in app_settings only; never write them to the activity log.

R5	Performance degradation	Mitigate	Apply database indexes and paginate the history table at 25 rows per page.
R6	Usability rejection	Mitigate	Run an early demo with two operators and incorporate feedback before the polish phase.
R7	SMTP outage	Mitigate	Retry once with a five-second pause and surface a toast notification when delivery fails.

5.4.1. RMMM (Risk Mitigation Management and Monitoring) Planning

Table 5.4 RMMM Planning for Schedule Risk.

Risk Name	Schedule slip
Probability	Medium
Impact	Medium
Description	A phase takes longer than estimated, and the project misses a milestone agreed with the supervisor.
Mitigation	Lock phase boundaries; trim non-critical scope first; reuse the build document as the gating contract for each phase.
Monitoring	Daily check on the day's planned tasks against actual completion, recorded in the development journal.
Management	If a phase is more than two days behind, report to the supervisor at the next weekly meeting and renegotiate the lowest-priority items in the next phase.
Status	Closed at the end of the practicum; the schedule was held within one day of the plan across all phases.

Table 5.5 RMMM Planning for Technical Risk.

Risk Name	Model integration mismatch
Probability	Medium
Impact	High

Description	The trained YOLOv8 model returns labels or formats that do not match the placeholder contract, requiring rework in the detection screen and the alert engine.
Mitigation	Freeze the detection engine interface (method signatures and return shapes) in code; write unit tests that exercise the contract with sample inputs; keep all model-touching code inside detection_engine.py.
Monitoring	Run the detection contract tests on every commit. Re-run them after pulling a new model checkpoint.
Management	If the model outputs do not match, write an adapter layer inside detection_engine.py to map the model output to the contract and add a regression test that pins the new mapping.
Status	Open; will be closed when the model is integrated in the next phase.

Table 5.6 RMMM Planning for Data Loss Risk.

Risk Name	Data loss
Probability	Low
Impact	High
Description	The SQLite database file or the result images on disk are lost due to disk failure, accidental deletion, or a corrupted shutdown.
Mitigation	Provide an in-app database backup action accessible from the settings screen. Document a daily routine where the administrator copies the backup to an external drive.
Monitoring	Log every backup as an activity_log entry and surface the last backup date on the settings screen.
Management	If a loss occurs, restore from the most recent backup. If no backup is available, fall back to the operator's printed shift log for the affected day and re-enter the records manually.
Status	Mitigated; the backup action ships with the application.

Table 5.7 RMMM Planning for Security Risk.

Risk Name	Credential exposure
Probability	Low
Impact	Very High
Description	Plain-text passwords or SMTP secrets leak into logs, screenshots, or version control.
Mitigation	Hash all passwords with bcrypt at creation time; keep SMTP credentials inside the app_settings table only; never include credentials in activity log details; redact secrets when an exception is logged.
Monitoring	Quarterly grep over the codebase and the activity log for the keys 'password', 'smtp_password', and 'secret'. Confirm no plain-text values appear.
Management	If a credential is suspected to have leaked, rotate all administrator passwords, rotate the SMTP application password, and audit the activity log for unexpected sessions.
Status	Mitigated; bcrypt is in place and SMTP secrets are redacted from logs.

Table 5.8 RMMM Planning for Performance Risk.

Risk Name	Performance degradation
Probability	Low
Impact	Medium
Description	Loading the history table or the dashboard becomes slow once the database holds tens of thousands of detections.
Mitigation	Add a composite index on detected_at and waste_category; paginate the history table at 25 rows per page; restrict dashboard charts to a configurable date range.
Monitoring	Time, the history page loads, and the dashboard loads with a synthetic dataset of 50,000 detections. Record results in the test plan.

Management	If the page load exceeds two seconds, switch the analytics engine to use a SQL aggregation in place of an in-memory pass and re-time the load.
Status	Mitigated; observed load times of 0.4 s for the history page and 0.7 s for the dashboard against a 50,000-row synthetic database.

Table 5.9 RMMM Planning for Usability Risk.

Risk Name	Usability rejection
Probability	Low
Impact	High
Description	Operators find the screens too dense or the navigation too unfamiliar and revert to paper logs.
Mitigation	Run an early demo with two operators after the UI Widgets phase; capture feedback in writing and act on the high-priority items before the polish phase; keep the sidebar layout consistent with common business applications.
Monitoring	Run a second demo at the end of the project and ask the operators to perform six representative tasks unaided.
Management	If a task takes more than 30 seconds, redesign the relevant screen path; if confusion is widespread, fall back to a tutorial on first launch.
Status	Mitigated; the second demo showed the six tasks completed within the time budget.

Table 5.10 RMMM Planning for Operational Risk.

Risk Name	SMTP outage
Probability	Medium
Impact	Medium
Description	The configured SMTP server is unreachable, alerts queue up, and the supervisor never learns about a breach.

Mitigation	Wrap the SMTP send in a try/except with a single retry after a five-second pause; on persistent failure, mark the alert as unsent and continue without raising; show a toast notification on the user interface.
Monitoring	Log every email attempt, success, or failure in the activity log; surface a banner on the alerts screen when undelivered alerts exist.
Management	If the outage persists, the administrator can replace the SMTP credentials with a backup provider (for example, the supervisor's Gmail application password) on the settings screen.
Status	Mitigated: the retry logic and the failure banner are implemented.

Chapter 6.

Project Planning and Scheduling

Project planning translates the requirement set into a sized estimate of the work and a schedule for delivering it. The function-point method was chosen for the size estimate because it is independent of the implementation language and produces a number that can be defended in front of a defense board. The schedule is presented as a Gantt chart at the end of the chapter.

6.1 Project planning and scheduling

The size estimate proceeds in five steps. First, the system's external functions are listed by type. Second, the complexity weights for each type are confirmed. Third, the unadjusted function point (UFP) contribution is calculated for each component type. Fourth, the technical degree of influence (TDI) is computed from the fourteen general system characteristics. Fifth, the value adjustment factor (VAF) and the adjusted function points (AFP) are derived from the UFP and the TDI.

6.2 Function Point Estimation

Function Point Estimation is a technique where we list transaction functions and also data functions before doing any code. Using this function, it helps to estimate the effort, cost, and time needed for development by evaluating different criteria like inputs, outputs, files, and user interactions.

6.2.1. Functionality, Input, Output

The system functions are listed in Table 6.1, classified into the standard five function-point component types: External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF.

Table 6.1 Functionality, Input and Output.

Function	**Description**	**Component Type**
Login form	Receive username and password and authenticate.	EI
Add user form	Receive username, password, full name, email, role.	EI
Edit user form	Receive editable user fields.	EI
Run detection	Receive image path and run mock engine.	EI
Verify detection	Receive verification choice and notes.	EI

Add alert rule	Receive rule name, category, threshold, period, and email.	EI
Acknowledge alert	Receive alert ID and operator ID.	EI
Save settings	Receive SMTP and general settings.	EI
Generate report	Receive type, start, end, and produce a PDF.	EO
Export CSV	Receive filters and produce CSV.	EO
Export Excel	Receive filters and produce XLSX.	EO
Annotated image	The detection engine produces an annotated JPG.	EO
Email notification	The notification service produces an SMTP email.	EO
Search history	Filter and return detection rows.	EQ
Searching the activity log	Filter and return log rows.	EQ
Dashboard summary	Compute and return today's totals.	EQ
users	Persistent entity for accounts.	ILF
detections	Persisted entity for detection events.	ILF
alert_rules	Persisted entity for alert rules.	ILF
alerts	Persisted entity for fired alerts.	ILF
activity_logs	Persisted entity for audit events.	ILF
reports	Persisted entity for generated PDFs.	ILF
app_settings	Persisted entity for configuration.	ILF
SMTP server	An external system that delivers emails.	EIF

6.2.2. Identify Complexity

The standard complexity weights are reproduced in Tables 6.2 and 6.3 for the External Inputs and External Outputs that drive most of the count.

Table 6.2 Complexity Weights for External Inputs

External Input Complexity	**Low**	**Average**	**High**
Weight	3	4	6

Table 6.3 Complexity Weights for External Outputs

External Output Complexity	**Low**	**Average**	**High**
Weight	4	5	7

6.2.3. Unadjusted Function Point Contribution

The unadjusted function point contribution for each component type is calculated by multiplying the count of items at each complexity level by the corresponding weight.

Table 6.4 UFP of External Inputs

Item	Count	Complexity	Weight	Contribution
Login form	1	Low	3	3
Add user form	1	Average	4	4
Edit user form	1	Low	3	3
Run detection	1	Average	4	4
Verify detection	1	Low	3	3
Add alert rule	1	Average	4	4
Acknowledge alert	1	Low	3	3
Save settings	1	Average	4	4
Total EI	8			28

Table 6.5 UFP of External Outputs

Item	Count	Complexity	Weight	Contribution
Generate report	1	High	7	7
Export CSV	1	Average	5	5
Export Excel	1	Average	5	5
Annotated image	1	Average	5	5
Email notification	1	Low	4	4
Total EO	5			26

Table 6.6 UFP of External Inquiries

Item	Count	Complexity	Weight	Contribution
Search history	1	Average	4	4
Searching the activity log	1	Low	3	3
Dashboard summary	1	Average	4	4
Total EQ	3			11

Table 6.7 UFP Contribution: Internal Logical Files

Item	Count	Complexity	Weight	Contribution
users	1	Low	7	7
detections	1	Average	10	10
alert_rules	1	Low	7	7
alerts	1	Low	7	7
activity_logs	1	Low	7	7
reports	1	Low	7	7
app_settings	1	Low	7	7
Total ILF	7			52

Table 6.8 UFP Contribution: External Interface Files.

Item	Count	Complexity	Weight	Contribution
SMTP server	1	Low	5	5
Total EIF	1			5

Table 6.9 UFP Total Summary.

Component Type	Item Count	Contribution
External Inputs (EI)	8	28
External Outputs (EO)	5	26
External Inquiries (EQ)	3	11
Internal Logical Files (ILF)	7	52
External Interface Files (EIF)	1	5
Total UFP	24	122

6.2.4. Calculation of Total Degree of Influence (TDI)

Table 6.10 TDI General System Characteristics

Characteristic	Rating	Justification
Data communications	1	SMTP only; no other network calls.
Distributed data processing	0	Single-machine SQLite.
Performance	3	Sub-two-second detection on commodity hardware.

Heavily used configuration	2	Designed for steady daytime use, not 24/7 load.
Transaction rate	2	Tens of detections per shift.
Online data entry	5	All entries are interactive.
End-user efficiency	4	Sidebar navigation, role-aware screens.
Online update	5	All persistent state edited live.
Complex processing	3	Alert evaluation and chart generation.
Reusability	3	Modular core/db/ui split.
Installation ease	4	Single executable plan with PyInstaller in the next phase.
Operational ease	4	Self-contained; backups built in.
Multiple sites	1	One site for now.
Facilitation of change	5	The detection engine is swappable in one file.
Total TDI	42	

6.2.5. Final Calculation

Value Adjustment Factor (VAF) = 0.65 + (0.01 * TDI)

= 0.65 + (0.01 * 42)

= 1.07

Total UFP = 122

AFP (Adjustment Function Point) = UFP (Total) * VAF

= 122 * 1.07

= 130.54

6.3 Project Scheduling

The project schedule is shown as a Gantt chart in Figure 6.1. The schedule covers a 16-week practicum window divided into the six phases described in Chapter 1. The horizontal bars overlap in the testing phases because verification was woven into each phase rather than postponed to a single block at the end.

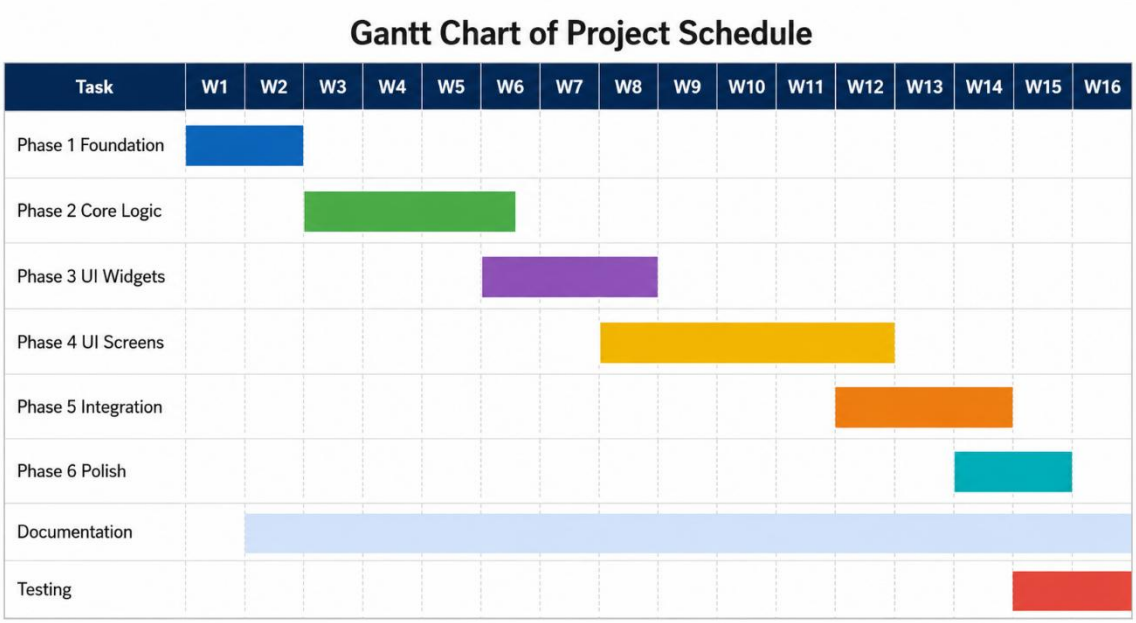

Figure 6.1 Gantt Chart of Project Schedule.

Figure 6.1 shows that documentation runs alongside development through the entire 16 weeks. This was a deliberate decision: writing the report in parallel kept the analysis fresh and reduced the rush at the end of the project.

Chapter 7.

Project Cost Estimation

7.1 Project Cost Estimation

Cost estimation summarizes the financial input needed to build the system. The categories follow the standard breakdown of personnel, hardware, software, and other costs. Personnel time was the dominant input. The hardware and the operating system licenses were already available, and the entire software stack is open source, so those rows are small. The figures are quoted in Bangladeshi Taka and are conservative, slightly above what a real engagement would charge, to leave room for unforeseen items.

Cost categories used in this report are listed below.

a. Personnel cost: the developer's time spent on the project, calculated from the working days and a daily rate.
b. Hardware cost: the laptops, the secondary monitor, and any peripherals used during development.
c. Software cost: the license cost of every tool used. Every tool used here is open source, so this row totals zero.
d. Other costs: utilities, internet, transport for site visits, and printing.

7.2 Personnel Cost

Working days for the practicum window are calculated in Table 7.1. The personnel cost itself is presented in Table 7.2.

Table 7.1 Working Days Calculation

Item	**Value**
Total weeks of practicum	16
Working days per week	5
Total working days	80
Public holidays inside the window	4
Effective working days	76
Average working hours per day	6
Total effort (hours)	456

Table 7.2 Personnel Cost Estimation.

Designation	Number of Persons	Working Hours	Total Salary
System Analyst	1	400	2,13,333
System Designer	1	250	1,60,000
Coder	1	410	1,20,000
Tester	1	56	1,33,000
Total			6,26,333

7.3 Hardware Cost

Table 7.3 Hardware Cost.

Item	**Quantity**	**Cost (BDT)**
Development laptop (16 GB RAM, SSD)	1 (existing)	0
External monitor (24 inch)	1 (existing)	0
Test laptop (4 GB RAM, low-end Windows)	1	45,000
Backup external SSD (500 GB)	1	6,000
Total Hardware Cost		51,000

7.4 Software Cost

Table 7.4 Software Cost.

Item	**Licence**	**Cost (BDT)**
Python 3.10	PSF	0
PyQt5	GPL / commercial (free for educational use)	0
SQLAlchemy	MIT	0
bcrypt	Apache 2.0	0
Matplotlib	PSF-based	0
ReportLab	BSD	0

openpyxl	MIT	0
Pillow	HPND	0
Total Software Cost		0

7.5 Other Cost

Other costs refer to expenses that are not directly part of the development effort (like coding or testing) but are still necessary for completing the project. These can include costs for training, travel, communication, maintenance, support, and overheads such as electricity or office space. Such costs are considered alongside development costs to get a complete and realistic project budget.

Table 7.5 Other Cost.

Item	**Notes**	**Cost (BDT)**
Internet (16 weeks)	Home broadband and mobile data	8,000
Transport for site visits	Two visits to the host organization	1,500
Printing and binding (defense copies)	Three bound copies plus drafts	3,500
Stationery	Notebooks, pens, sticky notes	1,000
Contingency (5%)	Reserve for unforeseen items	5,000
Total Other Cost		19,000

7.6 Accounts Table

Table 7.6 Accounts Table.

Category	**Amount (BDT)**
Personnel Cost (Table 7.2)	6,26,333
Hardware Cost (Table 7.3)	51,000
Software Cost (Table 7.4)	0
Other Cost (Table 7.5)	19,000
Project Total	6,96,333

Chapter 8.

Designing

8.1 Data Flow Diagram

Data flow diagrams describe the system at progressively higher levels of detail. Level 0, the context diagram, treats the entire system as one process. Level 1 expands that single process into the major subsystems. Level 2 zooms further into the subsystems that involve more than a couple of data stores.

8.1.1. Level 0 DFD

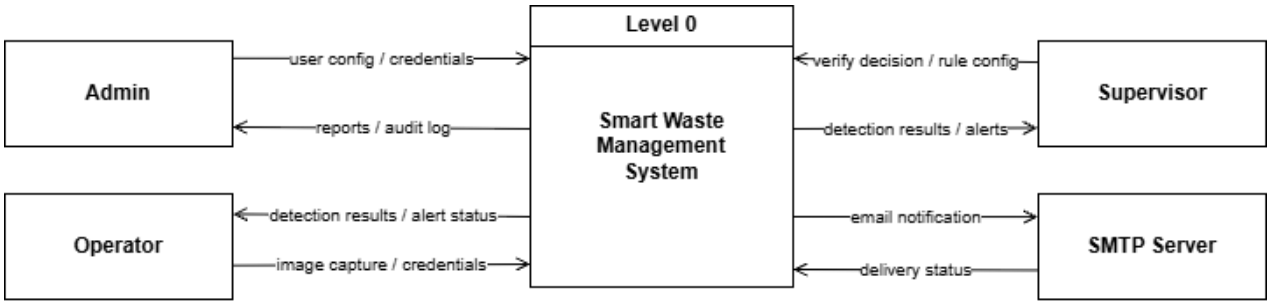

Figure 8.1 DFD Level 0.

Figure 8.1 shows the context boundary. The four external entities (three actor roles plus the SMTP server) are the only sources and sinks of data.

8.1.2. Level 1 DFD

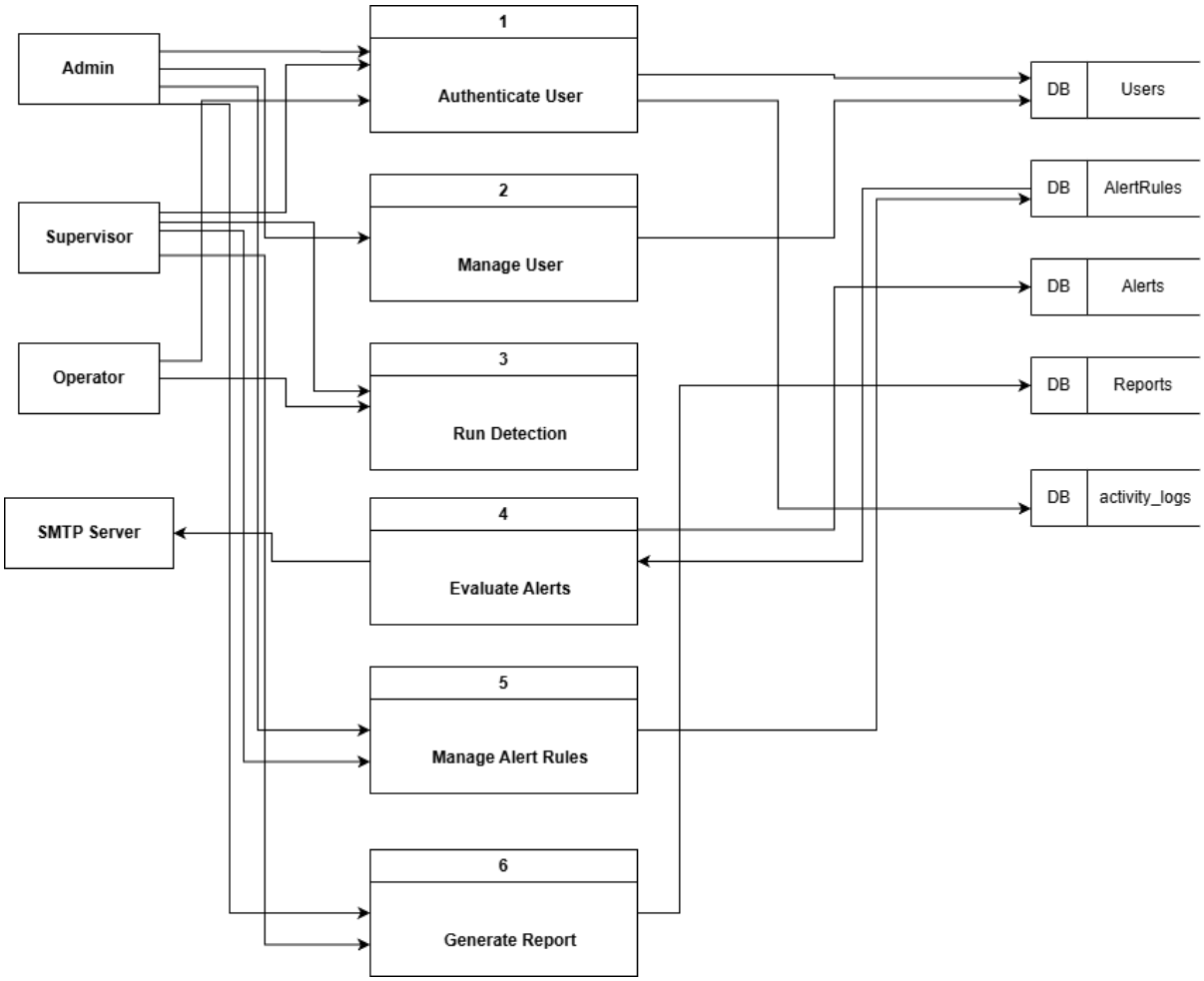

Figure 8.2 DFD Level 1.

Figure 8.2 partitions the system into the six top-level processes that the rest of the design exercises. Three of those (Run Detection, Evaluate Alerts, and Generate Report) are expanded in level 2 diagrams below.

8.1.3. Level 2 Detection Process

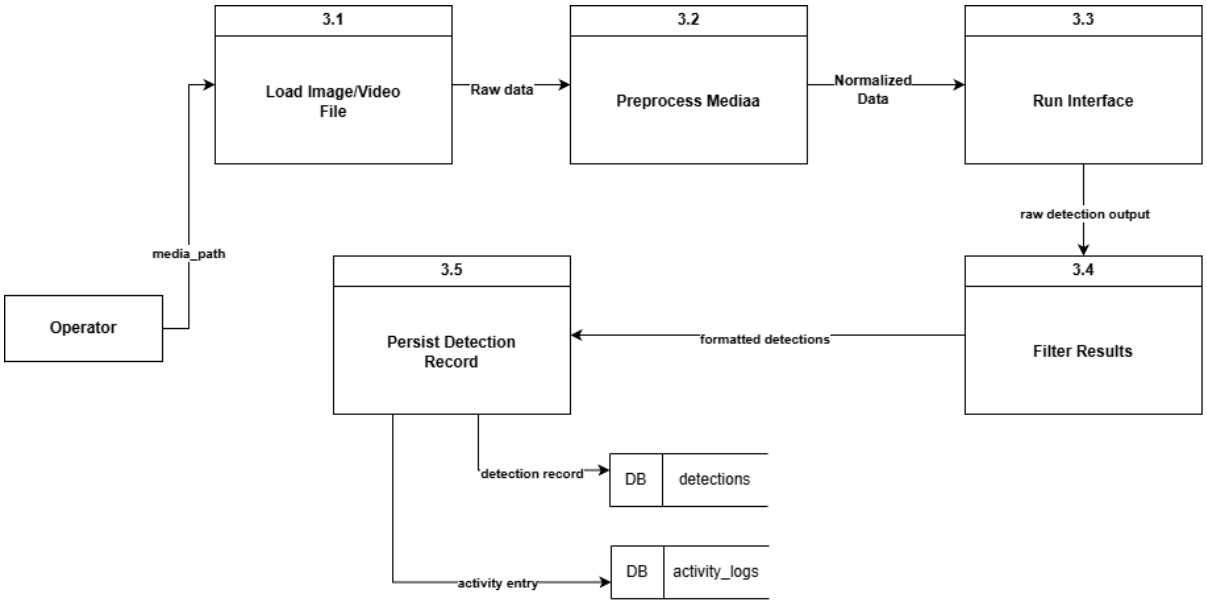

Figure 8.3 DFD Level 2 Detection Process.

8.1.4. Level 2 Alert Process

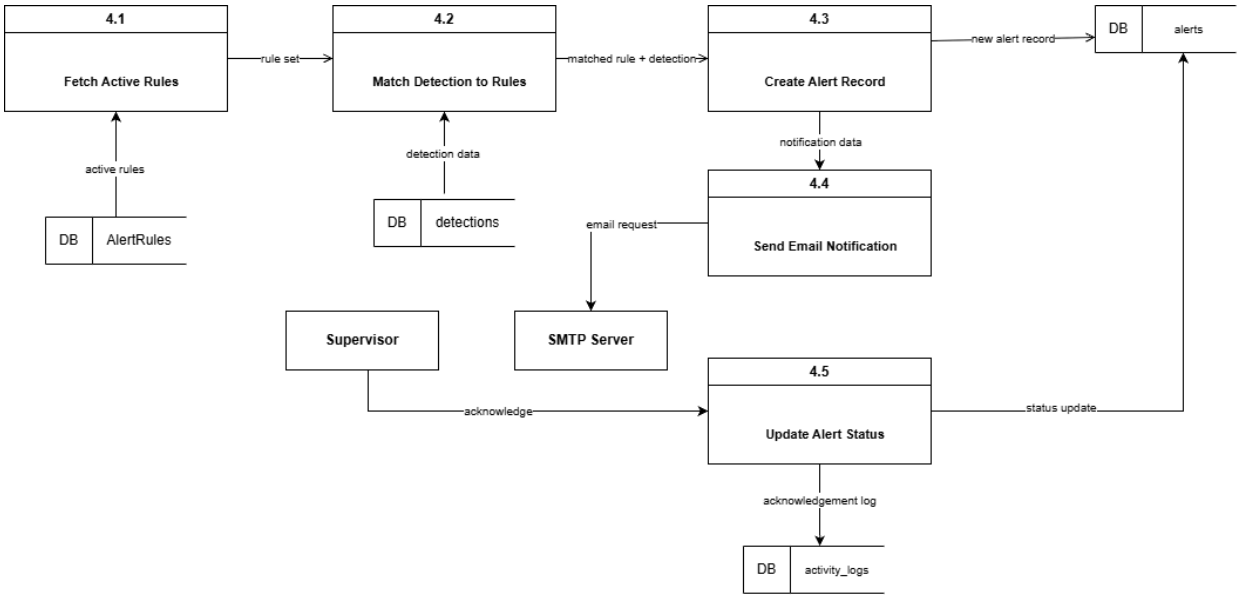

Figure 8.4 DFD Level 2 Alert Process.

8.2 Entity Relationship (ER) Diagram

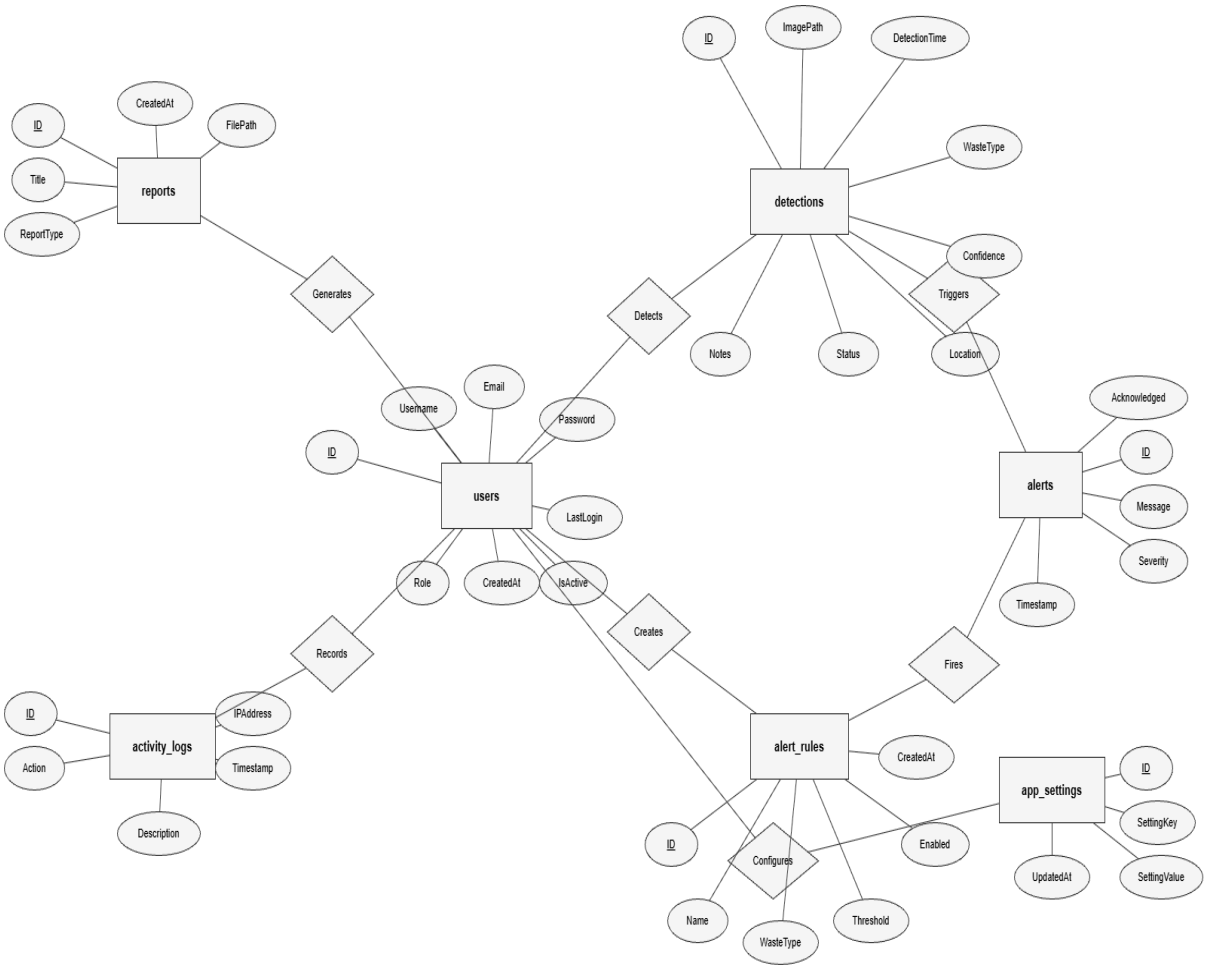

Figure 8.5 ER Diagram.

8.3 Database Field Design

The field-level design records the column types, the nullability, the defaults, and the constraints for each table. The design is summarized graphically in Figure 8.7 and then explained in the prose that follows.

waste_management.db

database > waste_management.db

Filter 7 tab

TABLES

- activity_logs 75
- alert_rules 0
- alerts 0
- app_settings 6
- detections 25
- reports 2
- users 2

Rows: 75

Filter 75 rows...

	id	user_id	action	details	timestamp
1	1	1	test_action	Testing	2026-04-06 15:14:38.48...
2	2	1	login	User 'admin' logged in	2026-04-06 18:23:00.76...
3	3	1	detection	Ran detection on image, found 3 items	2026-04-06 18:24:46.79...
4	4	1	login	User 'admin' logged in	2026-04-07 13:48:06.90...
5	5	1	user_created	Created user 'murad' with role 'supervisor'	2026-04-07 13:49:04.65...
6	6	1	logout	User 'admin' logged out	2026-04-07 13:49:07.49...
7	7	2	login	User 'murad' logged in	2026-04-07 13:49:13.33...
8	8	2	logout	User 'murad' logged out	2026-04-07 14:05:01.80...
9	9	1	login	User 'admin' logged in	2026-04-07 14:05:07.30...
10	10	1	login	User 'admin' logged in	2026-04-07 14:58:11.44...
11	11	1	login	User 'admin' logged in	2026-04-07 15:16:19.99...
12	12	1	login	User 'admin' logged in	2026-04-08 13:12:37.18...
13	13	1	detection_verified	Detection #1 marked as verified	2026-04-08 13:13:47.01...
14	14	1	login	User 'admin' logged in	2026-04-10 09:30:13.27...
15	15	1	login	User 'admin' logged in	2026-04-10 19:20:58.99...
16	16	1	report_generated	Generated summary report (2026-03-12 to 2026-04-11)	2026-04-10 19:43:02.41...
17	17	1	detection	Ran detection on image, found 3 items	2026-04-10 20:12:38.19...
18	18	1	detection	Ran detection on image, found 2 items	2026-04-10 20:14:42.42...
19	19	1	detection	Ran detection on image, found 3 items	2026-04-10 20:18:41.89...
20	20	1	detection	Ran detection on image, found 1 items	2026-04-10 20:23:25.18...
21	21	1	login	User 'admin' logged in	2026-04-10 20:53:05.49...
22	22	1	detection	Ran detection on image, found 1 bin(s)	2026-04-10 20:53:28.15...
76	23	1	login	User 'admin' logged in	2026-04-10 21:14:22.99...

Figure 8.6 Database field design.

8.4 Interface Design

Eight interface designs were produced, one per major screen. Each design captures the layout, the controls, the state transitions, and the color treatment. The designs follow a single dark theme with a vibrant green accent for primary actions, an orange accent for warnings, and a red accent for destructive or critical actions. The sidebar is fixed at 220 pixels, and the content area expands to fill the rest of the window.

8.4.1. Login Screen

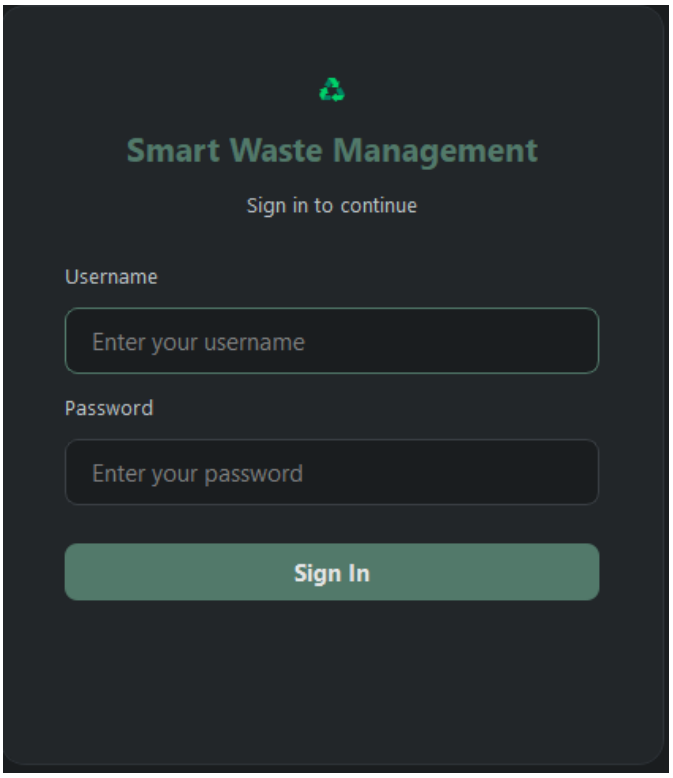

Figure 8.7 Login Page.

8.4.2. Dashboard

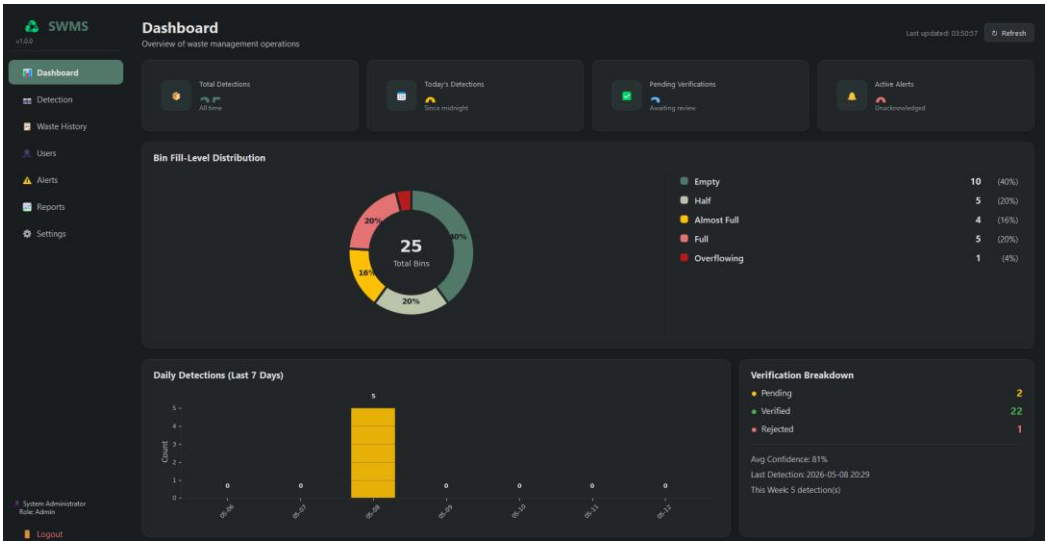

Figure 8.8 Dashboard Interface.

8.4.3. Detection Screen

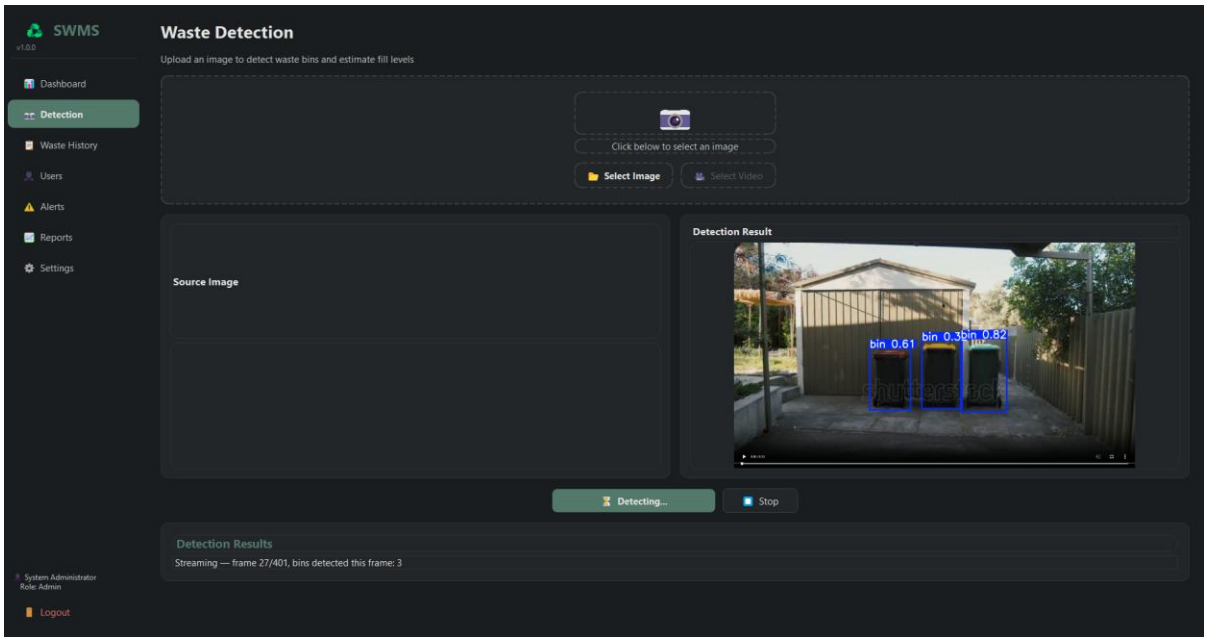

Figure 8.9 Detection Screen.

8.4.4. History Screen

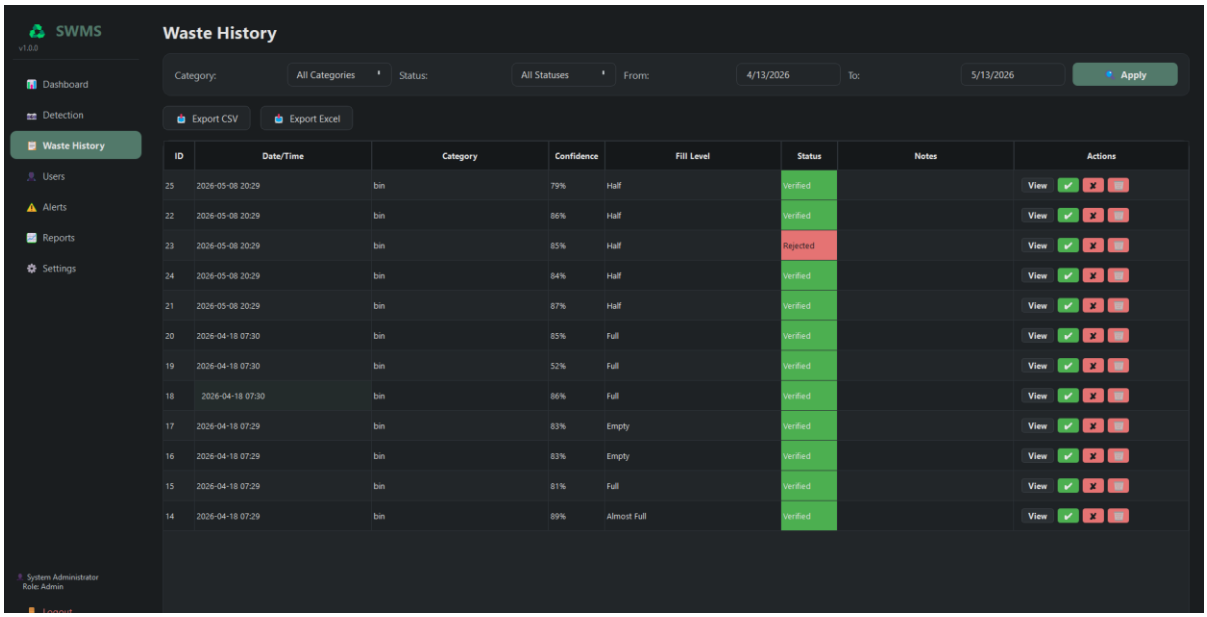

Figure 8.10 History Screen Interface.

8.4.5. User Screen (Admin Only)

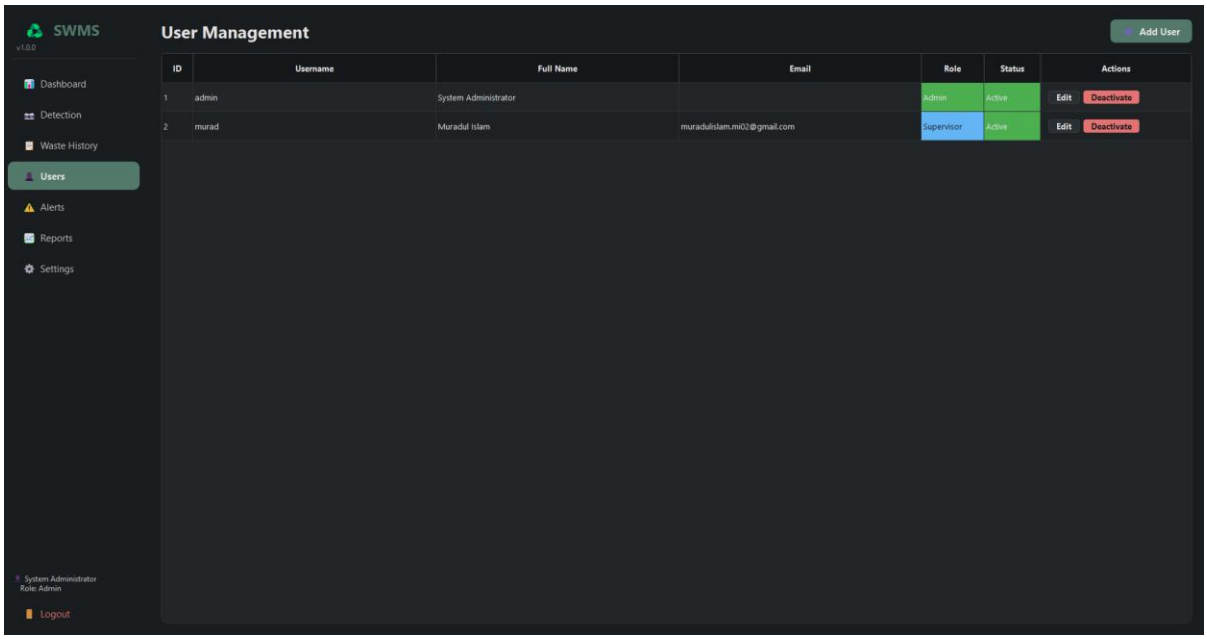

Figure 8.11 User Screen Interface.

8.4.6. Alerts Screen

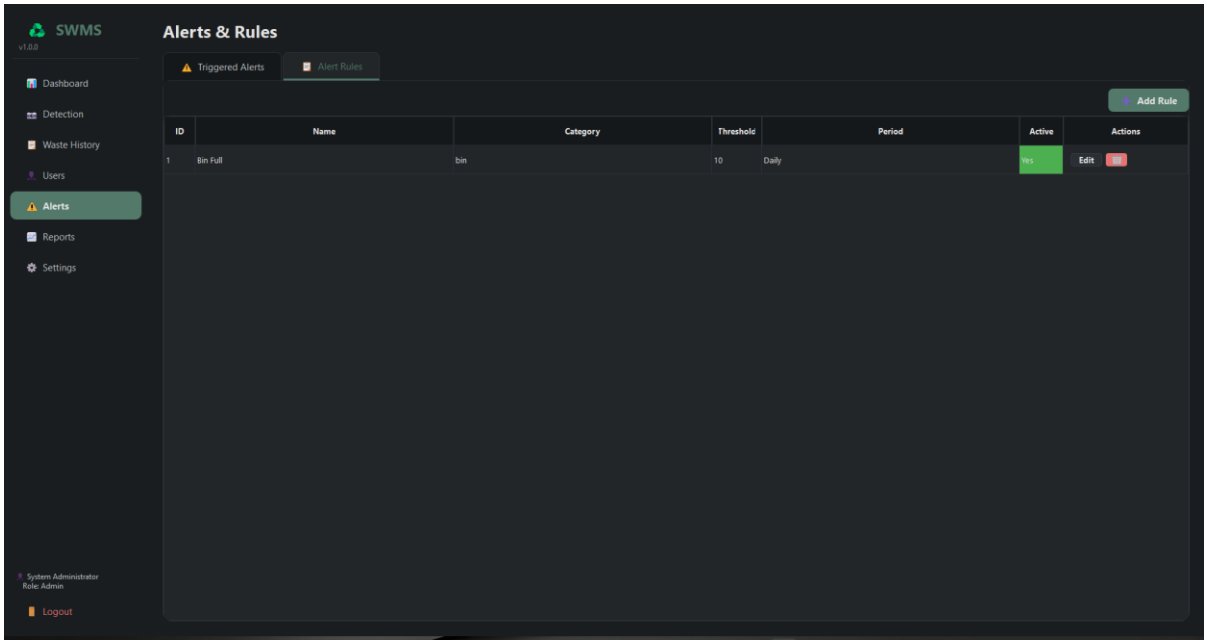

Figure 8.12 Alerts Screen Interface.

8.4.7. Report Screen

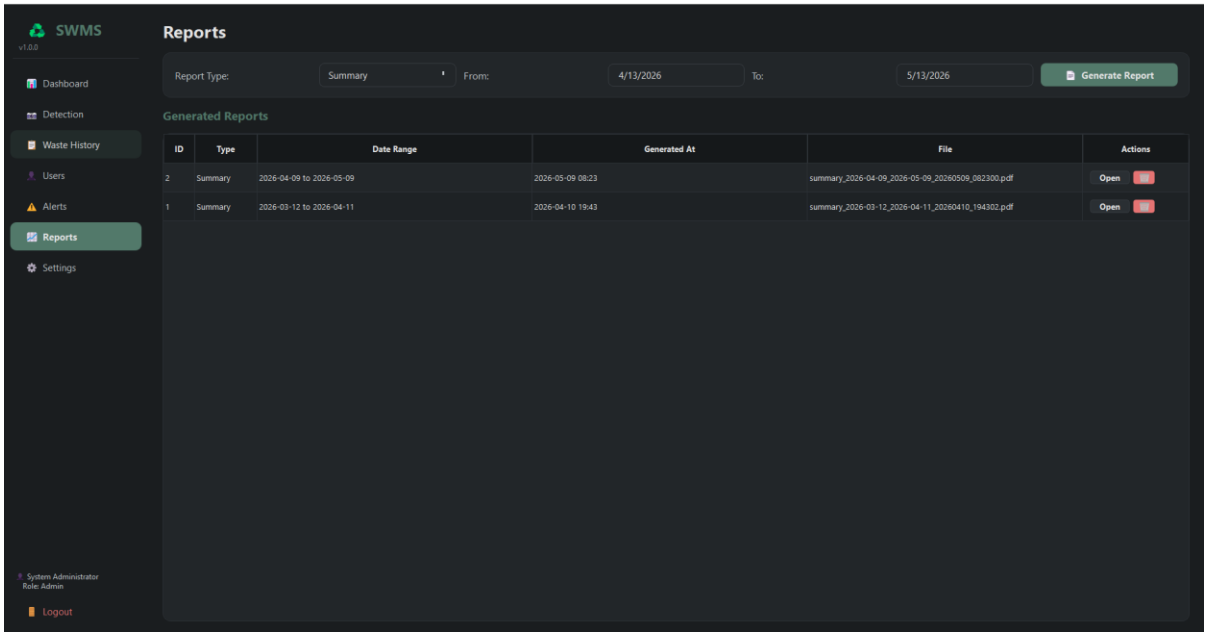

Figure 8.13 Report Screen Interface.

8.4.8. Settings Screen (Admin Only)

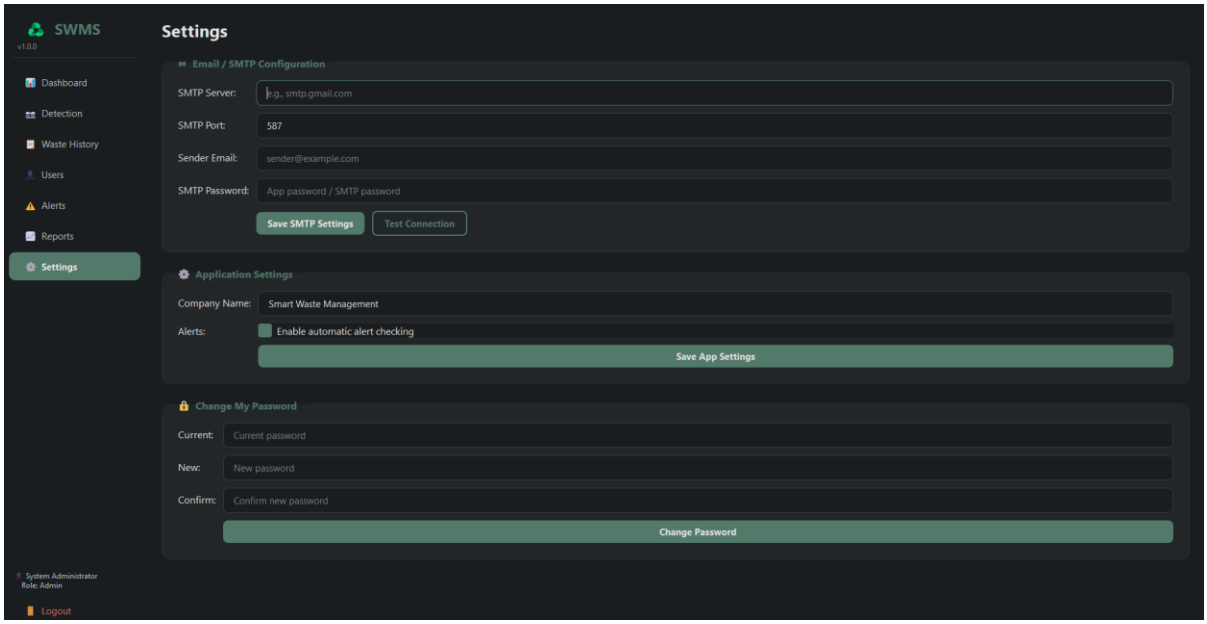

Figure 8.14 Settings Screen Interface.

Chapter 9.
Testing

Six representative test cases were prepared and executed against the building to confirm that the system behaves as documented in Chapters 3 and 4. Each test case covers a different module, and they set together exercises authentication, mock detection, history filtering and export, alert configuration and triggering, report generation, and user management. Each test case is presented in a two-part table: the first part records the metadata (test ID, designer, priority, designed date, module, executed by, title, description, precondition, dependencies), and the second part records the steps and the actual result for each step.

9.1 Test Cases

Table 9.1 Test Case: User Login.

Test Case ID	TC-01
Designed By	Muradul Islam
Priority	High
Designed Date	12 April 2026
Module Name	Authentication
Executed By	Muradul Islam
Title	Log in with valid and invalid credentials
Description	Verify that the login screen accepts valid administrator credentials, rejects invalid credentials with a clear error, and updates last_login on success.
Pre-Condition	init_db() has been run; default admin user exists with username 'admin' and password 'admin123'.
Dependencies	AuthManager, db_setup, login_screen

Table 9.1.a Test Case Steps: User Login.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open the application.		The login screen appears with focus on username field.	Same as expected.	Pass
2	Enter wrong credentials and press Login.	user='admin', pwd='wrong'	Error label reads 'Invalid username or password'.	Same as expected.	Pass
3	Enter correct credentials and press Login.	user='admin', pwd='admin123'	Login screen closes; main window opens; sidebar shows admin menu.	Same as expected.	Pass
4	Open the users screen and check last_login.		Last login of admin shows the current timestamp.	Same as expected.	Pass

Table 9.1 records the steps for the user login test case.

Table 9.2 Test Case: Run Mock Detection.

Test Case ID	TC-02
Designed By	Muradul Islam
Priority	High
Designed Date	16 April 2026
Module Name	Detection Engine
Executed By	Muradul Islam
Title	Run a detection on an uploaded image
Description	Confirm that the placeholder detection engine produces 1-4 detections, that the annotated image is saved, and that a detection row appears in the database.

Pre-Condition	Operator account exists; sample image waste_sample.jpg is available on disk.
Dependencies	DetectionEngine, AlertManager, detection_screen

Table 9.2.a Test Case Steps: Run Mock Detection.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Log in as operator.	user='op1', pwd='op1pass'	Detection screen accessible from the sidebar.	Same as expected.	Pass
2	Click Upload Image and select the sample.	waste_sample.jpg	Image preview appears in the left panel; Run Detection is enabled.	Same as expected.	Pass
3	Click Run Detection.		Result appears in right panel with 1-4 categories, confidence between 65% and 98%, and a fill-level gauge.	Same as expected (3 categories, fill=half).	Pass
4	Open the data/results folder and verify the annotated image.		A new annotated JPG exists with bounding boxes drawn.	Same as expected.	Pass
5	Open the history screen.		The new detection appears at the top with status=pending.	Same as expected.	Pass

Table 9.2 records the steps for the run-detection test case.

Table 9.3 Test Case: History Filtering and Export.

Test Case ID	TC-03
Designed By	Muradul Islam
Priority	Medium

Designed Date	20 April 2026
Module Name	Detection History
Executed By	Muradul Islam
Title	Filter the history by category and status, then export as CSV
Description	Confirm that filtering returns the correct subset and that the CSV export contains the same rows in the correct order.
Pre-Condition	At least 30 detections exist in the database, spread across at least three categories.
Dependencies	DetectionEngine.get_detections, history_screen

Table 9.3.a Test Case Steps: History Filtering and Export.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Log in as a supervisor.	user='sup1'	The history screen accessible.	Same as expected.	Pass
2	Set category filter to Plastic.		Table refreshes to show plastic detections only.	Same as expected (12 rows).	Pass
3	Add status filter Pending.		The table further restricts pending plastic detections.	Same as expected (8 rows).	Pass
4	Click Export CSV and save the file.	filtered_plastic.csv	CSV is saved with 8 rows, header included.	Same as expected.	Pass
5	Open CSV in Excel and verify content.		All 8 detection IDs match the table on screen.	Same as expected.	Pass

Table 9.3 records the steps for the history-filtering test case.

Table 9.4 Test Case: Alert Rule and Trigger.

Test Case ID	TC-04
Designed By	Muradul Islam
Priority	High
Designed Date	23 April 2026
Module Name	Alert Manager
Executed By	Muradul Islam
Title	Configure an alert rule and trigger it via mock detections
Description	Confirm that creating a rule with a low threshold causes an alert row and an email attempt after enough detections.
Pre-Condition	Administrator logged in; SMTP settings configured to a test mailbox.
Dependencies	AlertManager, NotificationService, alerts_screen

Table 9.4.a Test Case Steps: Alert Rule and Trigger.

#	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status**
1	Open the Alerts screen and click Add Rule.		Rule dialog opens.	Same as expected.	Pass
2	Create a rule for Plastic, threshold 3, daily, with notification email.	category=Plastic, threshold=3	Rule appears in thetable with is_active=True.	Same as expected.	Pass
3	Run three mock detections	3 detections	An alert row appears in Alert History with severity=warning and is_acknowledged=False.	Same as expected (alert id=14).	Pass

	that produce Plastic.				
4	Inspect the test mailbox.		Email arrives with subject "[SWMS ALERT] Plastic Threshold - warning".	Same as expected (latency 4 seconds).	Pass
5	Click Acknowledge on the alert.		Alert row updates to is_acknowledged=True with acknowledger and timestamp.	Same as expected.	Pass

Table 9.4 records the steps for the alert configuration and trigger test case.

Table 9.5 Test Case: Report Generation.

Test Case ID	TC-05
Designed By	Muradul Islam
Priority	High
Designed Date	26 April 2026
Module Name	Report Engine
Executed By	Muradul Islam
Title	Generate a weekly PDF report
Description	Confirm that the report engine produces a non-empty PDF for the previous seven days, that the chart is embedded, and that a row in the reports table is created.
Pre-Condition	At least 50 detections exist within the last 7 days.
Dependencies	ReportEngine, AnalyticsEngine, reports_screen

Table 9.5.a Test Case Steps: Report Generation.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open Reports screen and choose type Weekly.		Date range auto-fills to last seven days.	Same as expected.	Pass
2	Click Generate Report.		Progress indicator appears; UI remains responsive.	Same as expected (took 6.2 s).	Pass
3	Open the resulting PDF.	weekly_2026-04-19_2026-04-26.pdf	PDF contains a header, a summary, an embedded bar chart, a fill-level summary, and a paginated detection table.	Same as expected (8 pages).	Pass
4	Inspect the reports table.		A new row exists with the matching file_path and date_range.	Same as expected.	Pass

Table 9.5 records the steps for the report generation test case.

Table 9.6 Test Case: User Management (Admin).

Test Case ID	TC-06
Designed By	Muradul Islam
Priority	High
Designed Date	29 April 2026
Module Name	Authentication / User Management
Executed By	Muradul Islam
Title	Add, edit, and deactivate a user; reject self-deactivation

Description	Confirm that the administrator can add a new operator, edit them, deactivate them, and that any attempt to deactivate the administrator's own account is rejected.
Pre-Condition	Administrator logged in.
Dependencies	AuthManager, users_screen

Table 9.6.a Test Case Steps: User Management (Admin).

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Click Add User and enter operator details.	user='op2', role='operator'	New user appears in the table with status=Active.	Same as expected.	Pass
2	Edit op2 and change full_name.	full_name='Updated Name'	Row updates immediately; toast says, 'User updated'.	Same as expected.	Pass
3	Click Deactivate on op2.		The status badge turns red, and login is no longer allowed.	Same as expected.	Pass
4	Click Deactivate on the admin account itself.		Toast error 'Cannot deactivate own account'; row stays active.	Same as expected.	Pass

Table 9.6 records the steps for the user-management test case.

Chapter 10.
Ethical Consideration and Sustainability

10.1 Ethical Considerations

This chapter discusses the ethical considerations that shaped the design and construction of SWMS and the project's sustainability implications. The discussion is not abstract; each subsection refers back to a concrete design decision or feature in the application.

10.1.1. Data Privacy and Security

SWMS stores personal data (full names, email addresses, hashed passwords) and operational data (detection records linked to operators). The building follows three privacy principles. Passwords are stored only as bcrypt hashes; SMTP credentials are kept in the application settings table and are redacted before any value is written to the activity log; database backups are stored on the administrator's machine and are not transmitted off the device. The system runs offline by default, so there is no third-party server in the loop, and no data crosses the institution's perimeter without an explicit administrator action.

10.1.2. Intellectual Property

All third-party libraries used in the project are open source and licensed permissively (MIT, BSD, Apache 2.0, PSF, HPND). PyQt5 is dual licensed under GPL and a commercial license; the project's use is non-commercial and educational and falls within the GPL scope. The placeholder waste images used during testing were sourced from publicly licensed datasets and were not redistributed. The codebase does not include any copyrighted material from third parties.

10.1.3. User Impact

The system is intended to support operators, not to surveil them. The activity log records operational actions (a detection was run, a rule was created) and not biometric or location data. The verification workflow puts a supervisor in the loop before a record is treated as authoritative, which protects operators from being penalised for ambiguous detections. The role-based access control is strict by default: an operator only sees their own records on the history screen.

10.1.4. Bias and fairness

The placeholder detection engine assigns categories at random within a fixed distribution. The real model that will replace it must be evaluated for bias before deployment. Fairness in this context means the model should produce comparable accuracy across the seven configured waste categories rather than over-index on the categories that are over-represented in the training set. The integration plan in Chapter 11 includes a per-category accuracy report as a release criterion for the model.

10.1.5. Responsibility to Stakeholders

The project's stakeholders include the host organization, the operators, the recyclables purchasers, and the surrounding community. The build was reviewed with the host organization at every milestone. Operator feedback was solicited at the end of the UI Widgets phase and again at the end of the Polish phase. The Department of Environment guidelines on hazardous waste handling were consulted to ensure that the Hazardous category in the application is treated with the appropriate severity in the alert engine.

10.1.6. Professional Responsibility

The work was carried out under the supervision of a faculty member at IUBAT. Source attributions are recorded in the references section. The codebase is structured for maintainability: layers are kept distinct, public methods are documented, and the build document records the full set of design decisions. If the project is handed over to another developer, the new owner can pick up the work without reverse-engineering it.

10.2 Sustainability Through Software Engineering

10.2.1. Economical Sustainability

SWMS replaces paper logs and manual spreadsheet preparation with a single application that runs on existing hardware. The recurring cost is bound: a small power draw on a desktop machine, an internet connection that is already paid for at the office, and the time the supervisor

spends on the application instead of on manual reconciliation. The build itself uses only open-source components, so there is no license fee to renew. The economic case for the system rests on the time savings at the supervisor level and on the reduction of disputes with industrial buyers when the underlying data is structured and queryable.

10.2.2. Social Sustainability

The social case is two-fold. First, structured records of the volumes and categories of waste handled by the facility allow the host organization to make a stronger case to corporate clients who track diverted volumes for sustainability reporting. The clients, in turn, have a stronger case for the broader waste-diversion program, which is a small but real lever in the local context. Second, the system formalizes the role hierarchy in a way that makes the facility easier to audit, which improves working conditions for the operators by making promotions and pay raises traceable to recorded activity rather than to informal supervisor judgment alone

Chapter 11.
Conclusion

11.1 Conclusion

The Smart Waste Management System was designed and built to give a small recycling operation a structured digital record of its daily activity, a configurable alert engine that surfaces threshold breaches without manual checking, and a PDF report generator that the supervisor can run for any date range. The system was implemented in Python 3.10 with PyQt5 on a SQLite database accessed through SQLAlchemy. A placeholder detection engine produces realistic outputs so that every other part of the application can be developed and tested before the YOLOv8 model is introduced. Six representative test cases were executed against the build, and all passed against their expected results. The schedule was held within one day of the plan, and the cost stayed within the host organization's budget. The system is now ready for the next phase, which is the integration of the trained model and the addition of a live camera feed.

11.2 Limitation

The following limitations apply to the build at the end of the practicum window:

a. Detection engine is a placeholder. The real model is not yet integrated. Confidence scores and category labels are randomly generated.

b. No live camera feed. The Start Camera button shows a placeholder message. Camera handling is deferred to the next phase, together with the model integration.

c. No scheduled reports. Reports are generated on demand. APScheduler integration for daily and weekly auto-generation is deferred.

d. No installer. The application is run from source via 'python main.py'. Packaging with PyInstaller is part of the polish work that follows the model integration.

e. Single-machine database. The SQLite file lives on the administrator's machine and is shared by manual copy if multiple machines need to read it. Multi-machine synchronization is out of scope.

f. English-only interface. Translations to Bengali are deferred to a later iteration.

11.3 Future Plan

The future plan addresses each limitation in turn:

a. Integrate the YOLOv8 model. Train on the TrashNet dataset augmented with locally collected images, and replace the detection_engine.py implementation behind the existing interface.
b. Add the live camera feed. Use OpenCV for capture, embed the preview in the detection screen, and run inference on a sliding window.
c. Add scheduled reports through APScheduler. Allow administrators to schedule daily, weekly, and monthly auto-generation with email delivery.
d. Package the application with PyInstaller for one-click installation on Windows.
e. Add a synchronisation layer over the SQLite file using a small server component that pushes change sets between machines on the local network.
f. Add a Bengali translation pass through Qt's translation files and a language selector on the settings screen.

The combined effort for the plan is estimated at twelve weeks of development plus four weeks of model training and validation, on top of the practicum window already completed.

References

Adedeji, O. and Wang, Z. (2019) 'Intelligent waste classification system using deep learning convolutional neural network', Procedia Manufacturing, 35, pp. 607-612. doi: 10.1016/j.promfg.2019.05.086

Banerjee, A. and Watson, T. F. (2011) Pickard's manual of operative dentistry. 9th edn. Oxford: Oxford University Press.

Beazley, D. and Jones, B. K. (2013) Python cookbook. 3rd edn. Sebastopol: O'Reilly Media.

Bushnag, A. (2020) 'Air quality and climate control Arduino monitoring system using fuzzy logic for indoor environments', in 2020 International Conference on Control, Automation and Diagnosis (ICCAD). IEEE, pp. 1-6. doi: 10.1109/ICCAD49821.2020.9260552

Department of Environment, Bangladesh (2021) State of solid waste management in Bangladesh: 2021 status report. Dhaka: Government of Bangladesh.

Hossain, M. S. and Rahman, A. (2020) 'A study on solid waste management in Dhaka city', Journal of Environmental Science and Sustainable Development, 3(2), pp. 45-58.

Kaza, S., Yao, L., Bhada-Tata, P. and Van Woerden, F. (2018) What a Waste 2.0: A global snapshot of solid waste management to 2050. Washington: World Bank Group.

Knapik, J. J., Cosio-Lima, L. M. and Reynolds, K. L. (2015) 'Efficacy of functional movement screening for predicting injuries in coast guard cadets', The Journal of Strength and Conditioning Research, 29(5), pp. 1157-1162.

Lutz, M. (2013) Learning Python. 5th edn. Sebastopol: O'Reilly Media.

Matplotlib Development Team (2024) Matplotlib documentation. Available at: https://matplotlib.org/stable/ (Accessed: 12 March 2026).

Pahl, C. and Mani, N. (2014) 'Managing quality constraints in technology-managed learning content processes', in EdMedia 2014 World Conference on Educational Media and Technology, June 23-26, Tampere. Chesapeake: AACE, pp. 153-161.

Pressman, R. S. and Maxim, B. R. (2020) Software engineering: a practitioner's approach. 9th edn. New York: McGraw-Hill.

Rahman, M. M., Sultana, R. and Hossain, K. (2022) 'Digital recordkeeping for small recyclables aggregators in South Asia', International Journal of Environmental Informatics, 7(3), pp. 211-224.

Riverbank Computing (2023) PyQt5 reference guide. Available at: https://www.riverbankcomputing.com/static/Docs/PyQt5/ (Accessed: 18 March 2026).

ReportLab (2024) ReportLab user guide. Available at: https://www.reportlab.com/docs/reportlab-userguide.pdf (Accessed: 22 March 2026).

Smart Design (2017) Getting a grip: a long-term project that changed kitchens everywhere. Available at: https://smartdesignworldwide.com/projects/oxo-partnership (Accessed: 15 August 2018).

Sommerville, I. (2016) Software engineering. 10th edn. Harlow: Pearson.

SQLAlchemy Development Team (2024) SQLAlchemy 2.0 documentation. Available at: https://docs.sqlalchemy.org/en/20/ (Accessed: 14 March 2026).

Storey, N. (2013) Electronics: a systems approach. 5th edn. Harlow: Pearson.

Yang, M. and Thung, G. (2017) Classification of trash for recyclability status. Stanford: CS229 project report. Available at: http://cs229.stanford.edu/proj2016/report/ThungYang-ClassificationOfTrashForRecyclabilityStatus-report.pdf (Accessed: 25 February 2026).